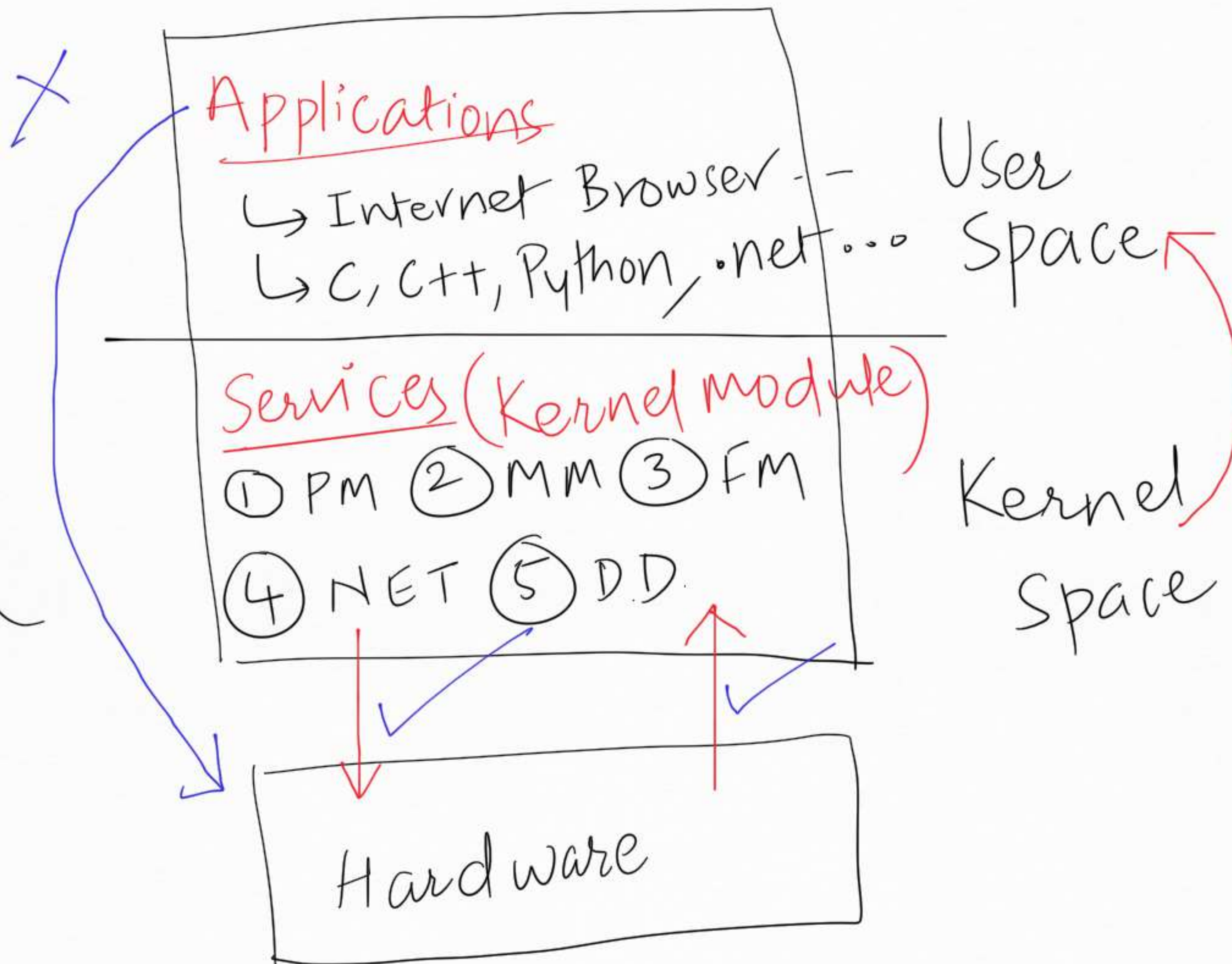


Ubuntu
O.S.



init / system d

Zoom APPⁿ

User

Kernel

1. Net

4. PM

2. D.D.

3. M.M

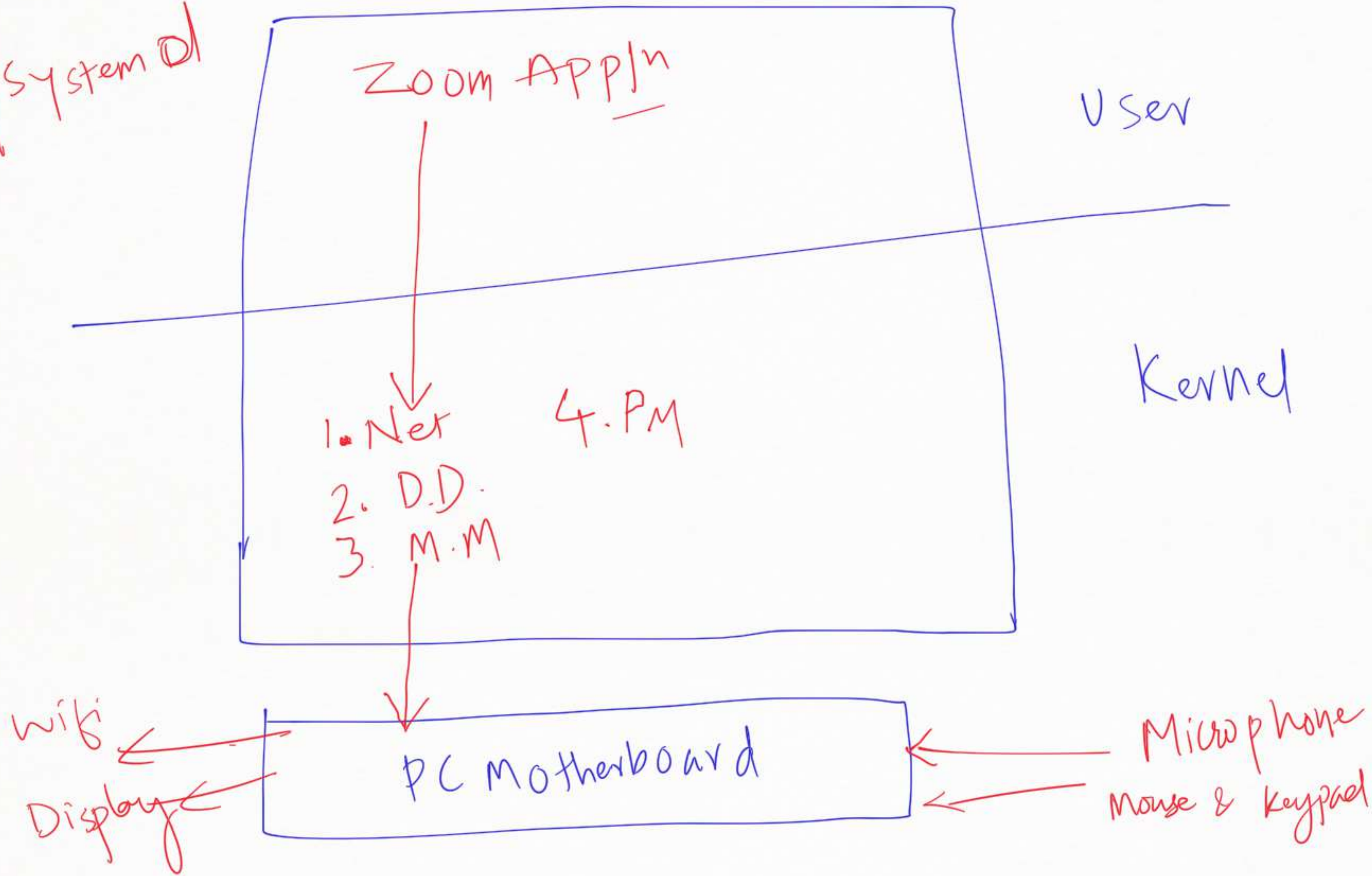
wiki

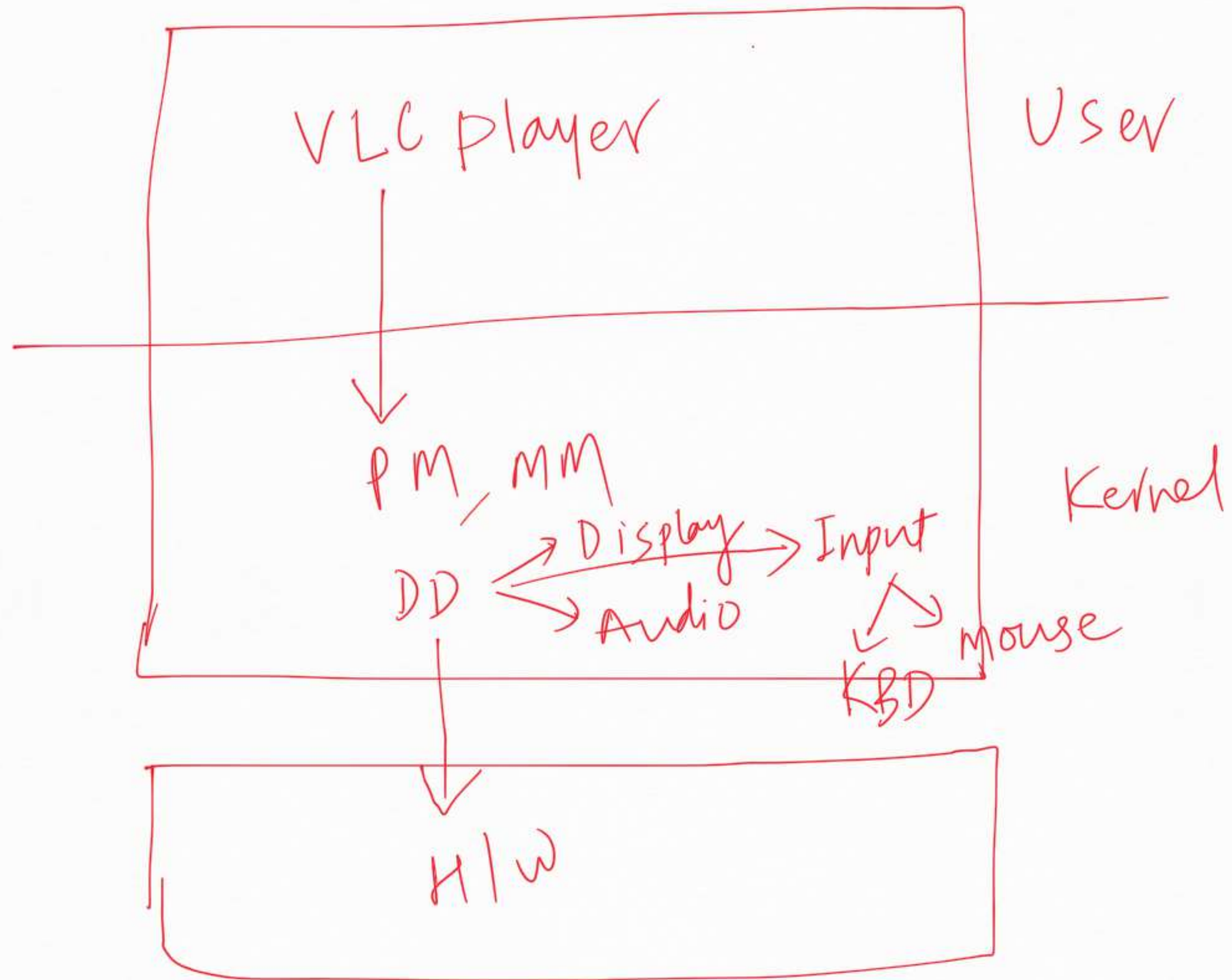
Display

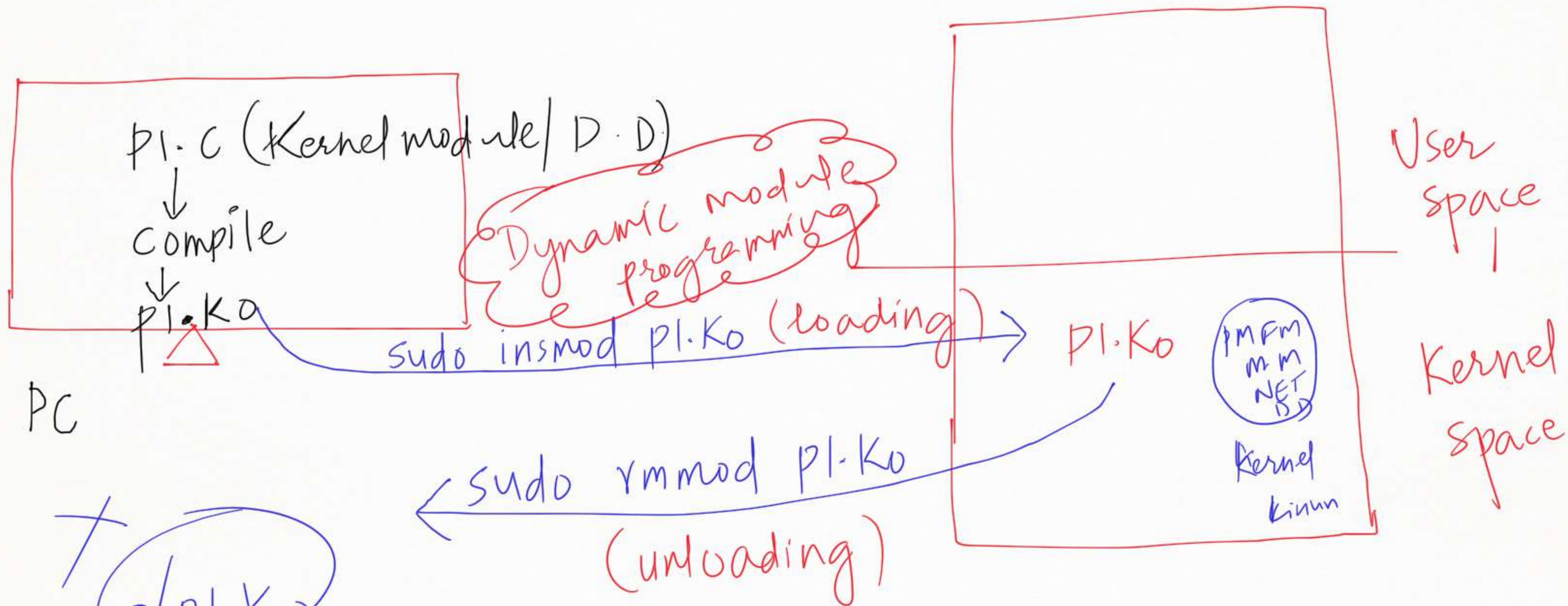
PC Motherboard

Microphone

Mouse & Keypad







~~`pl.ko`~~

`./a.out`

Power ON



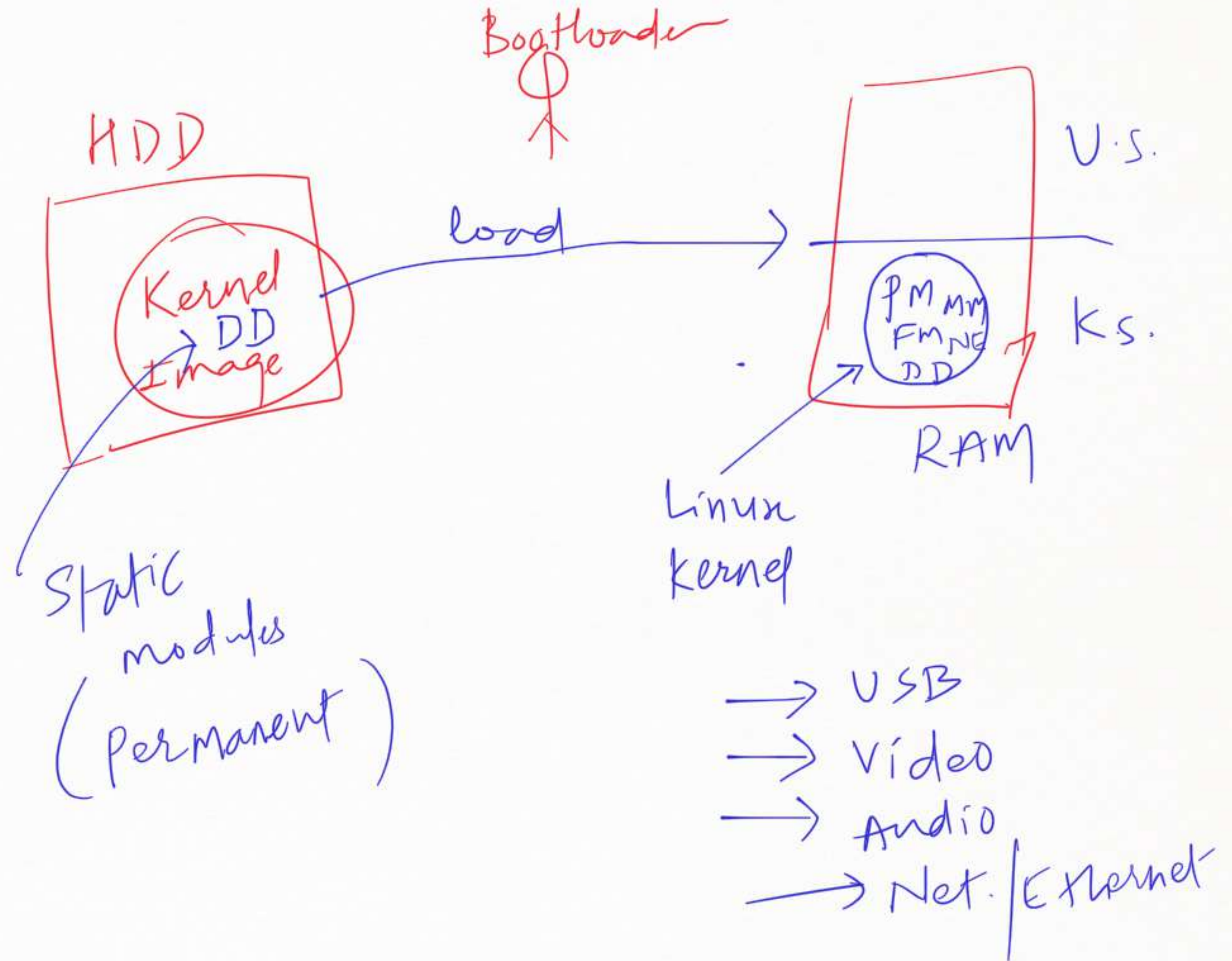
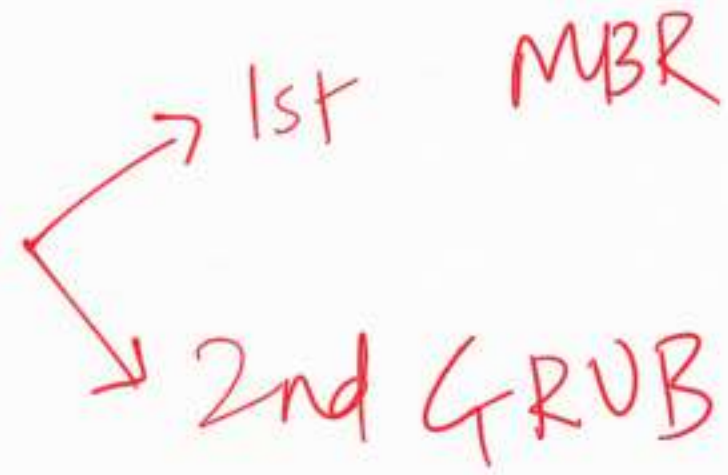
BIOS

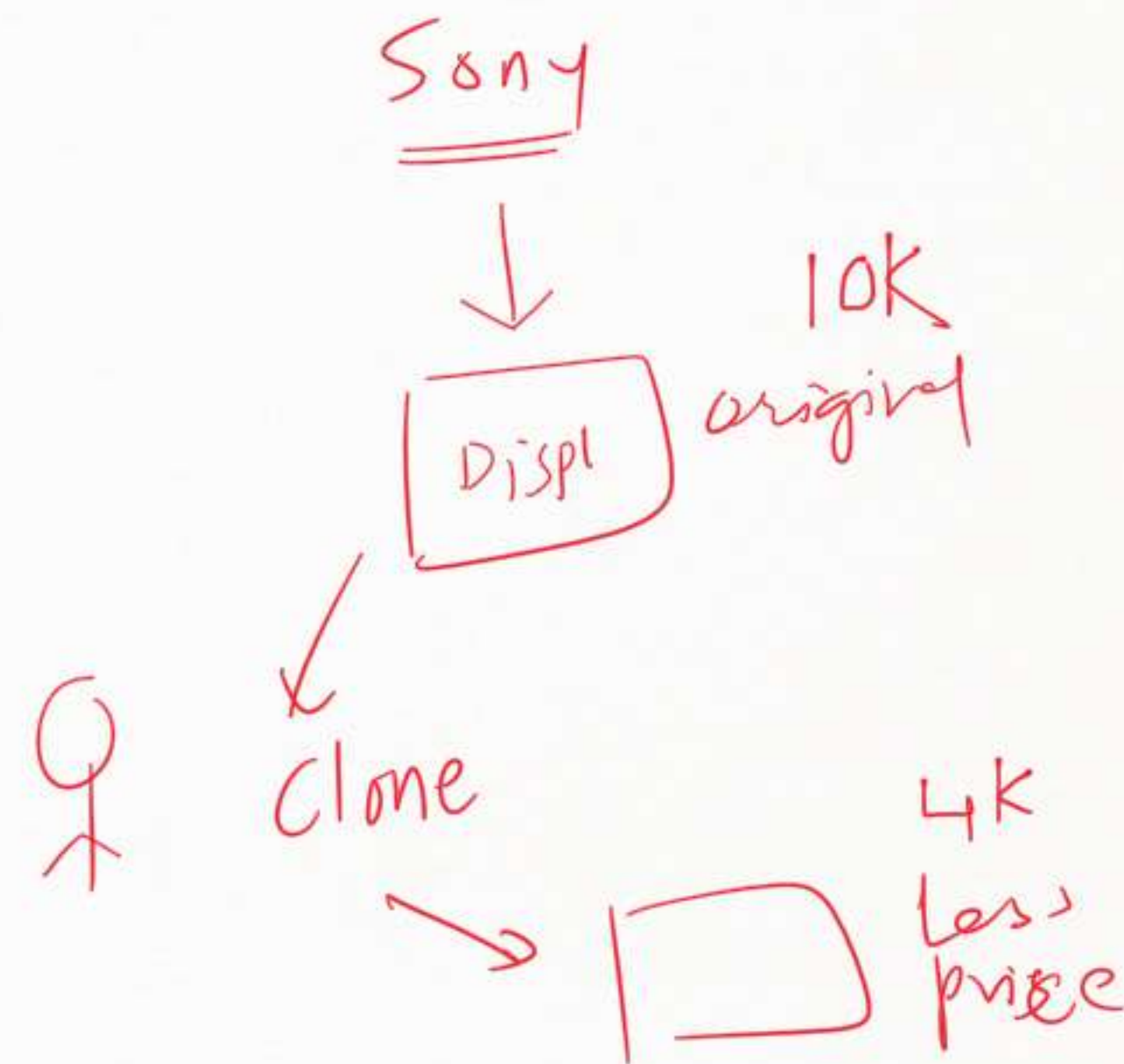
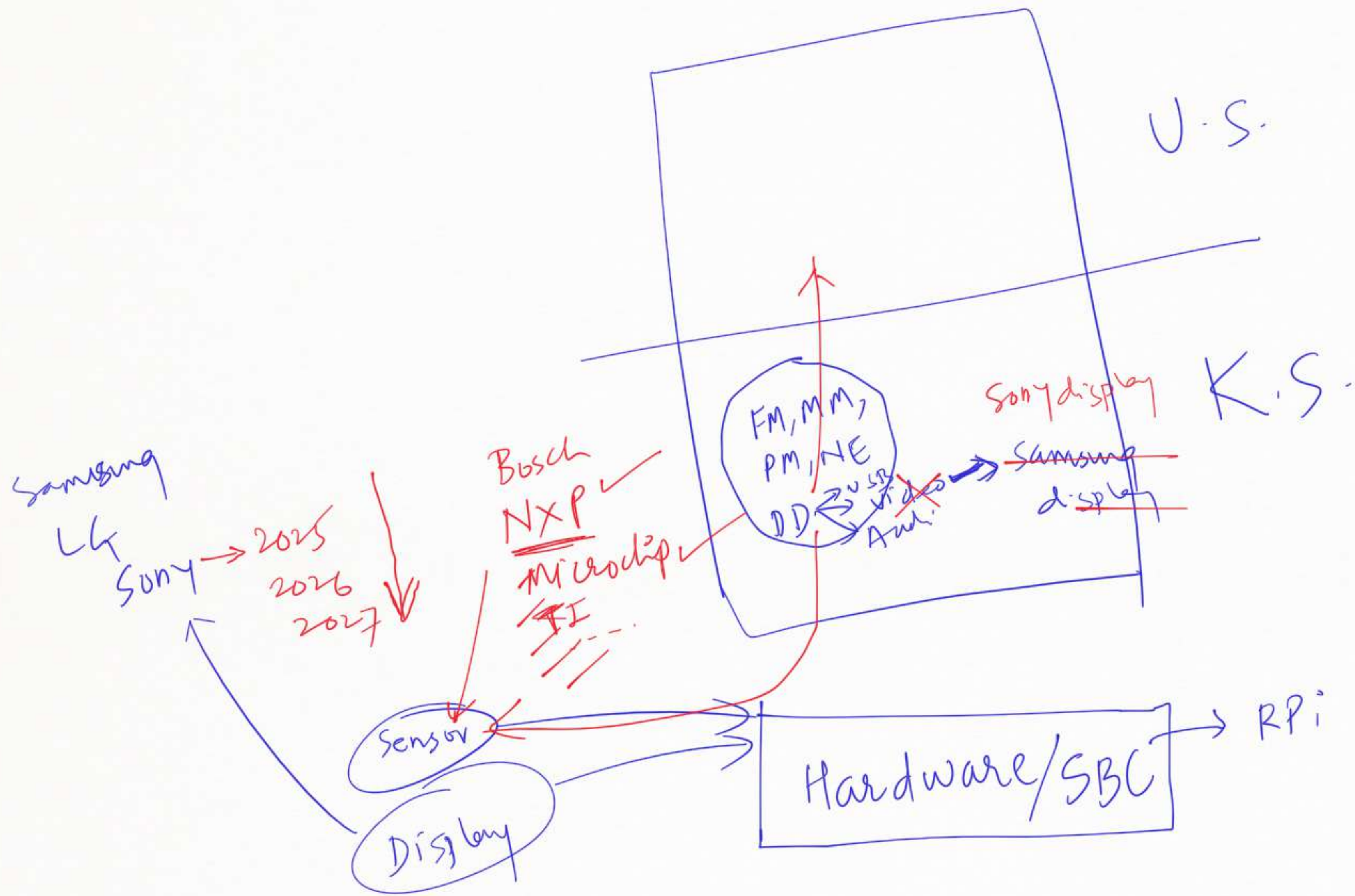


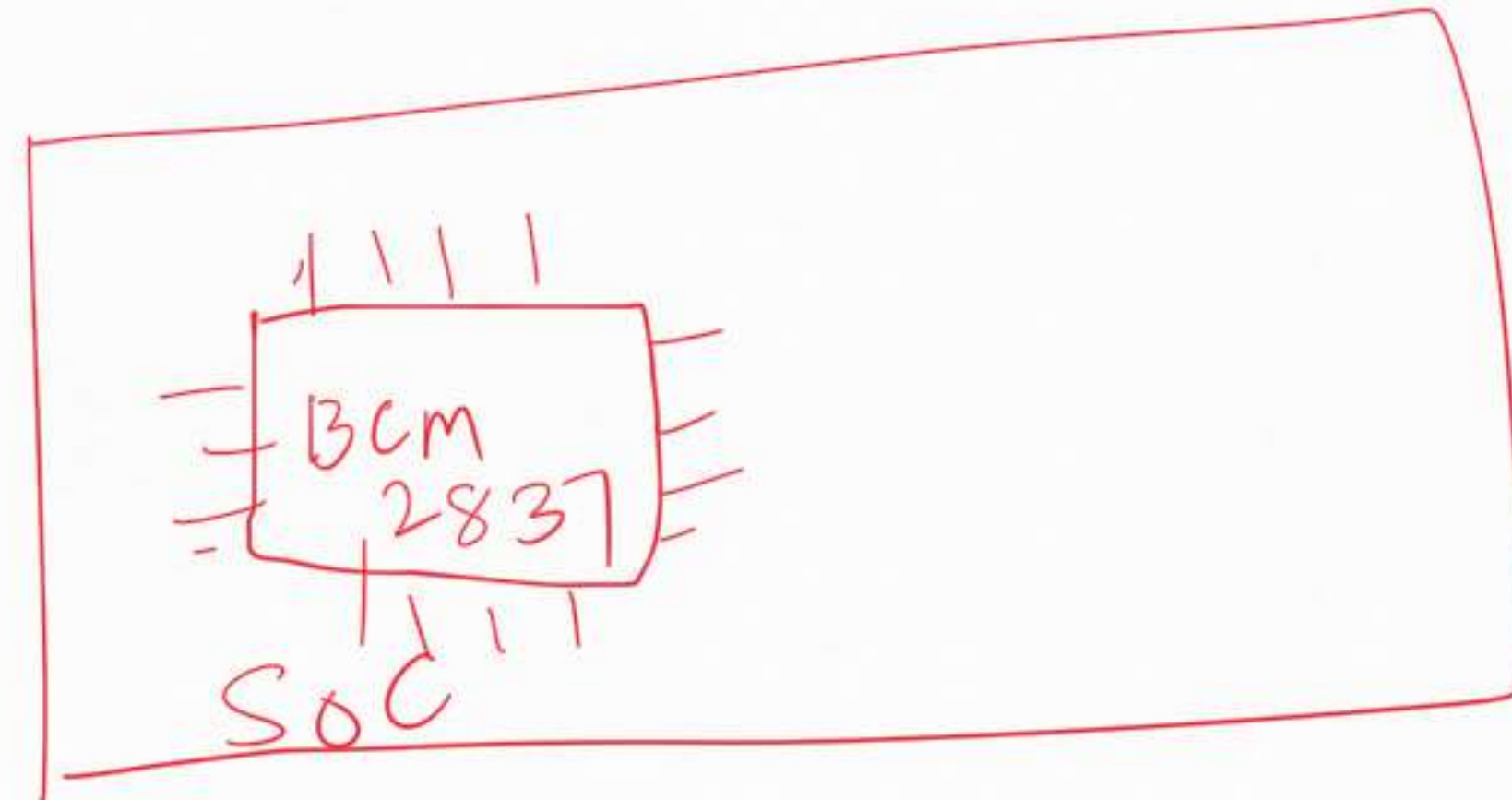
Bootloader



Kernel



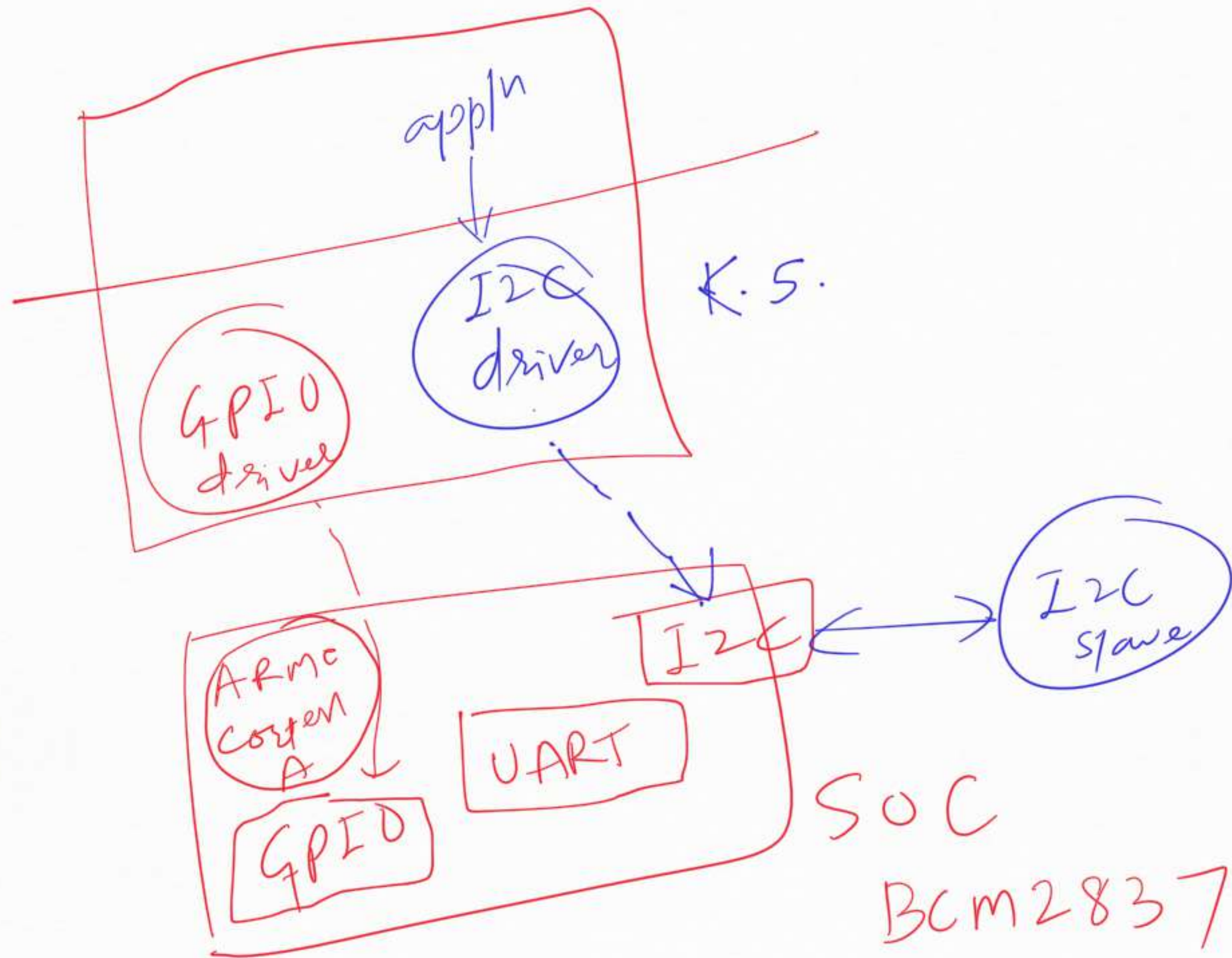


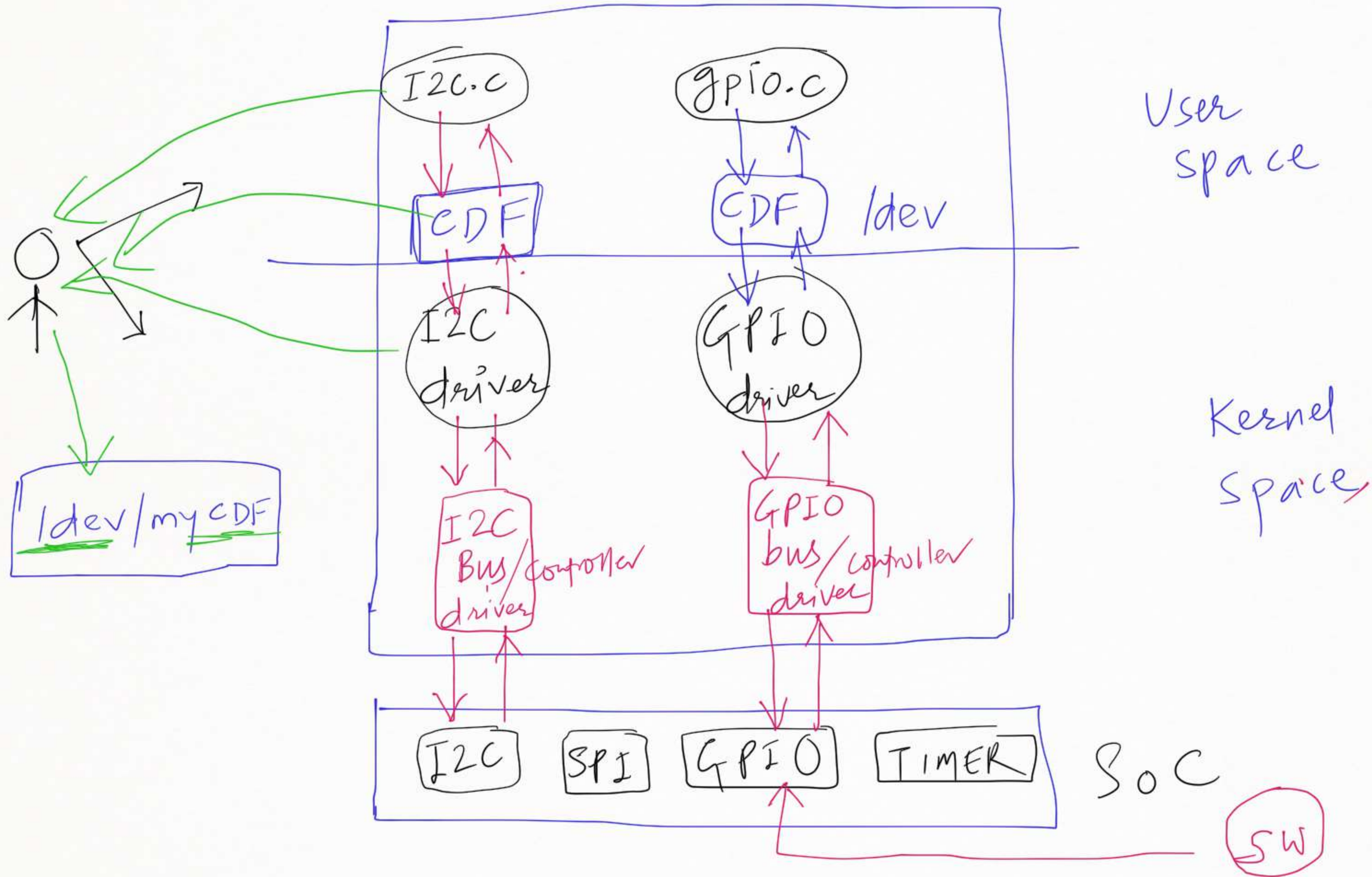


SBC

eg: RPi3B

Raspbian
O.S.
(Linux)



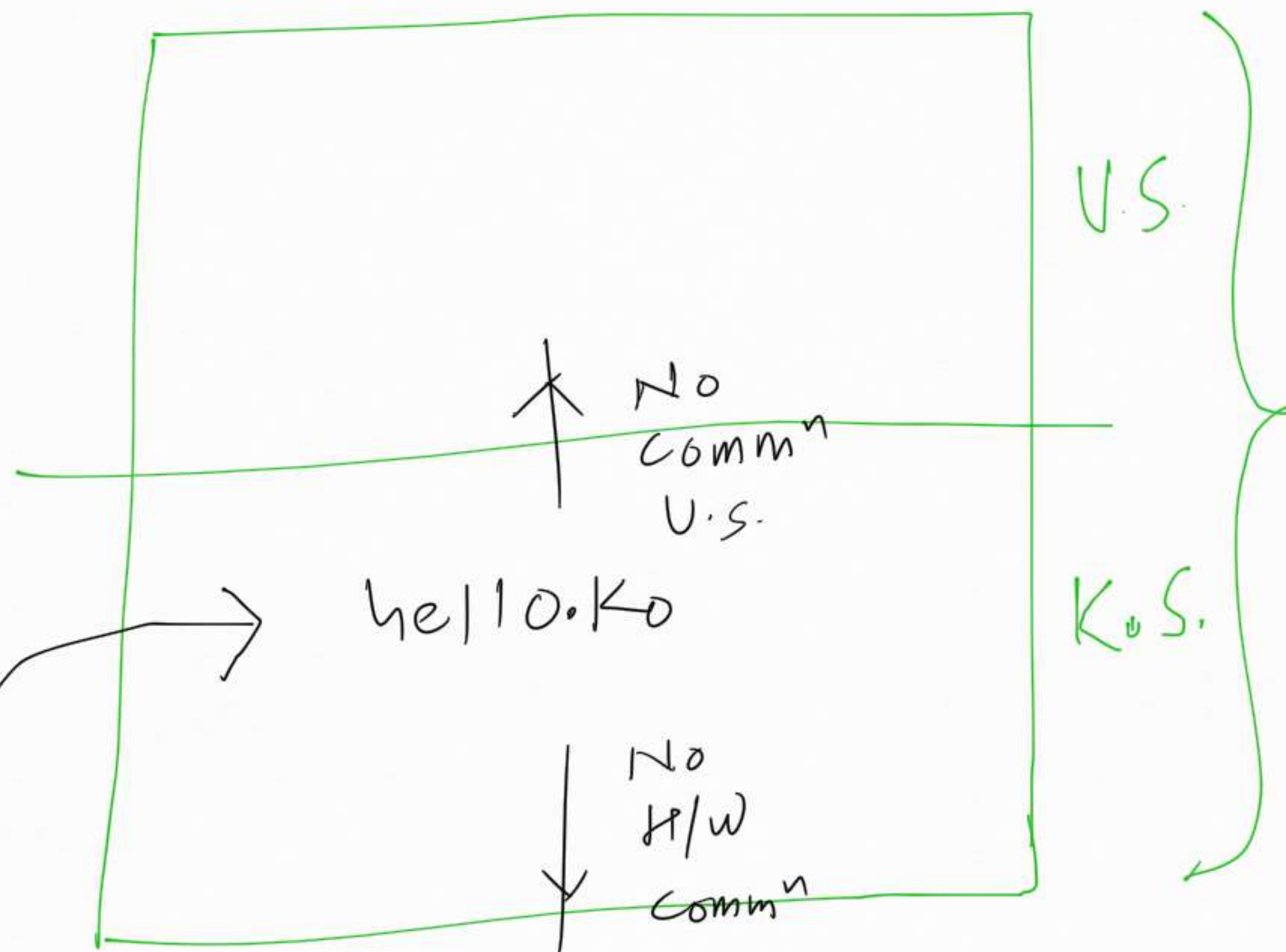


hello.c (P.D.)

↓
Compile
(build)

↓
hello.ko

↓
Kernel object



cleanup()
or
exit() }

1. free DMA
2. Unregister driver

}

init() {

1. DMA

2. Register device driver with Kernel

}

hello.ko insmod hello.ko
 ↗
 ↘
 rmmv hello.ko
 ↖

hello.ko → init() {
 ==
 {
 }

exit()
 ==
 }

K.S.

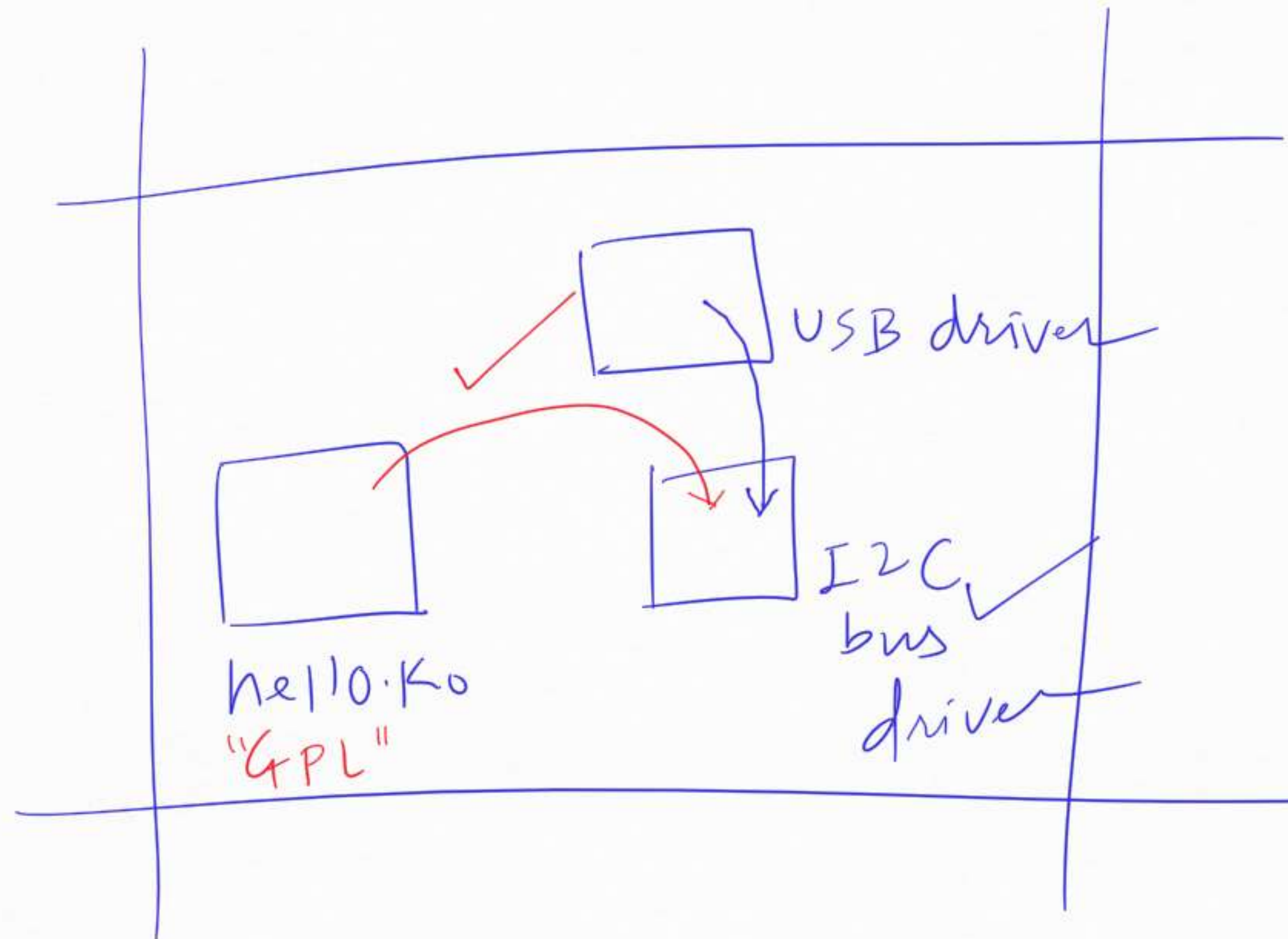
hello.c



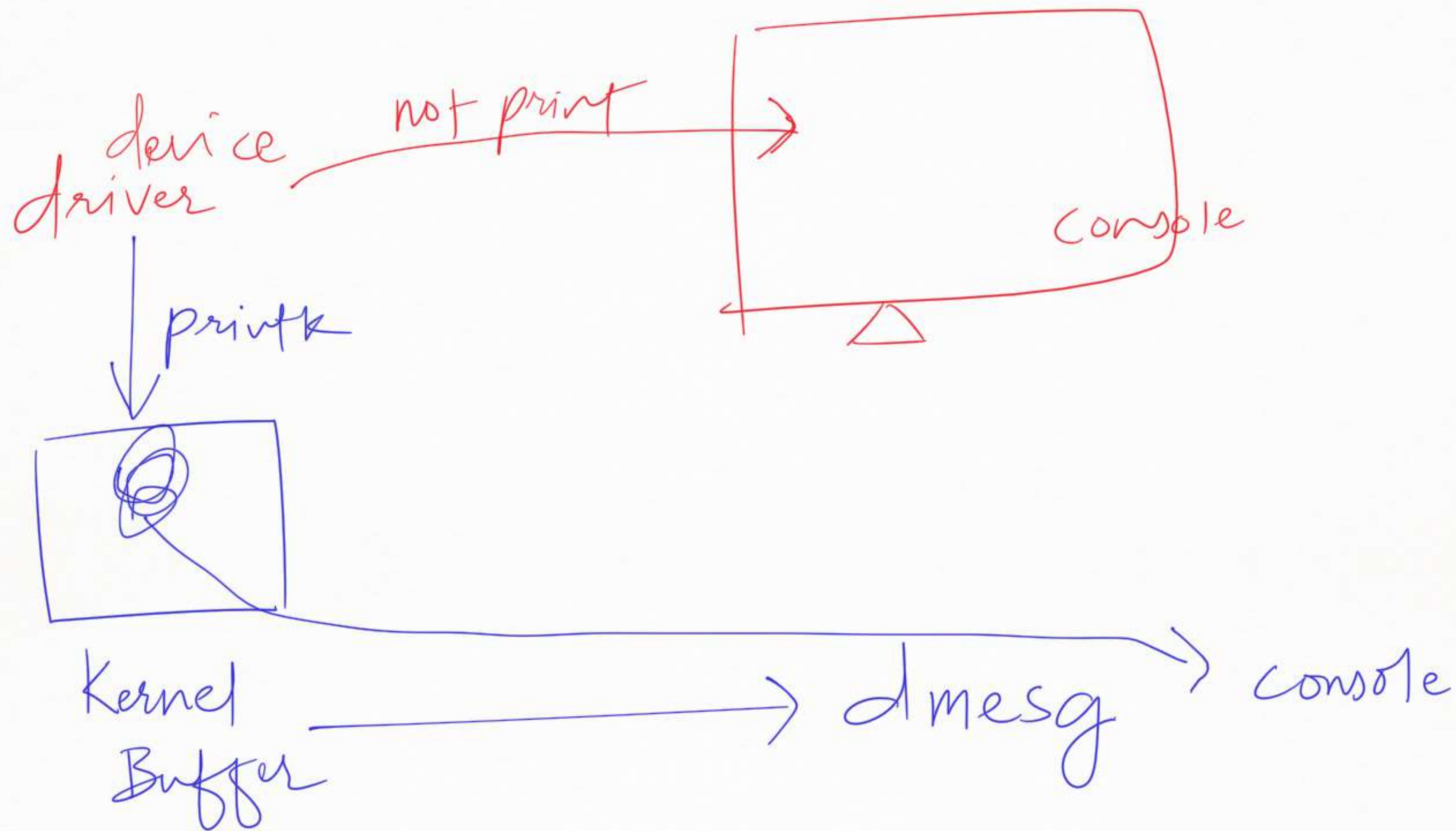
make (build your Kernel module)



hello.ko



K.S.



insmod

rmmod

lsmod

dmesg

cat /proc/devices

modinfo

/lib/modules/kernel-ver/build/~~linux~~
/Linux

/usr/include

C + DS, Linux U.S., Embc + Project

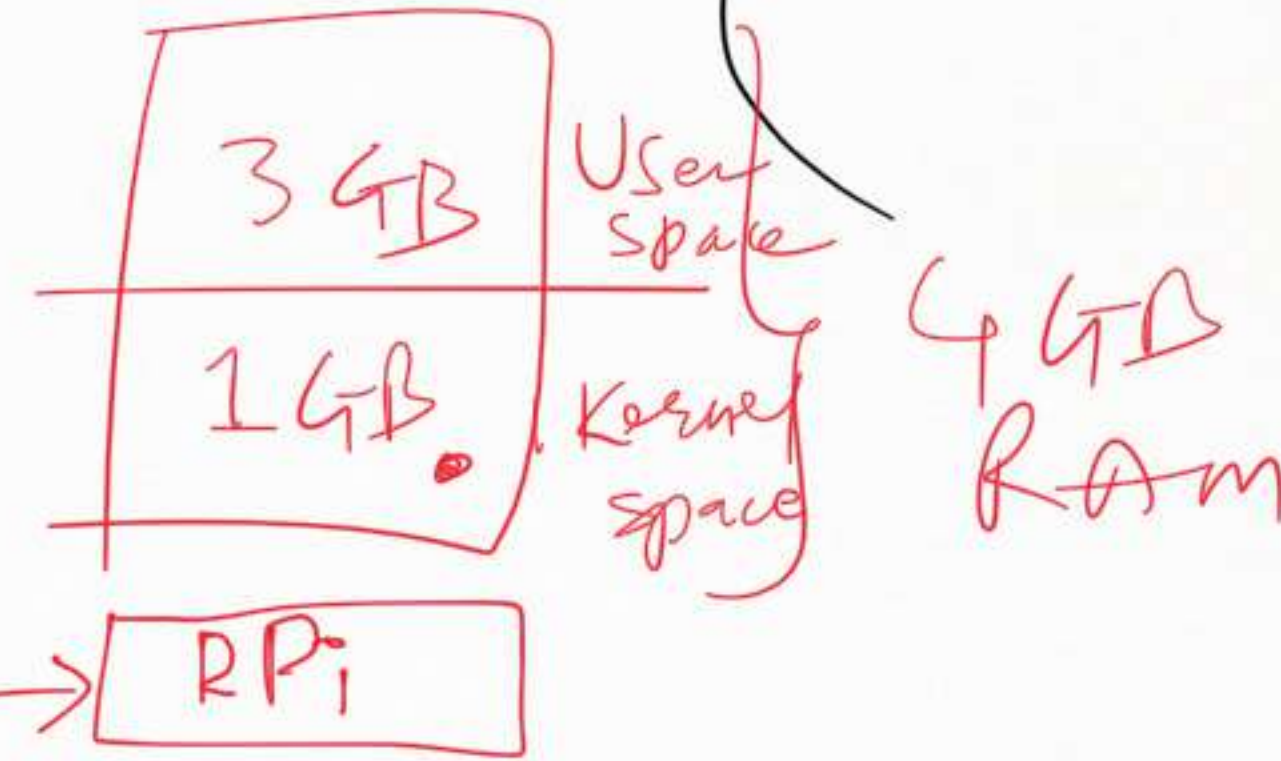
H/w: Raspberry Pi 3B / RPi5

hello world

Kernel module

Device Driver

(D) →
LED



O.S.

1. m/r card

2. USB pendrive

hello.c (D.D.)

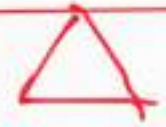


Build/compile

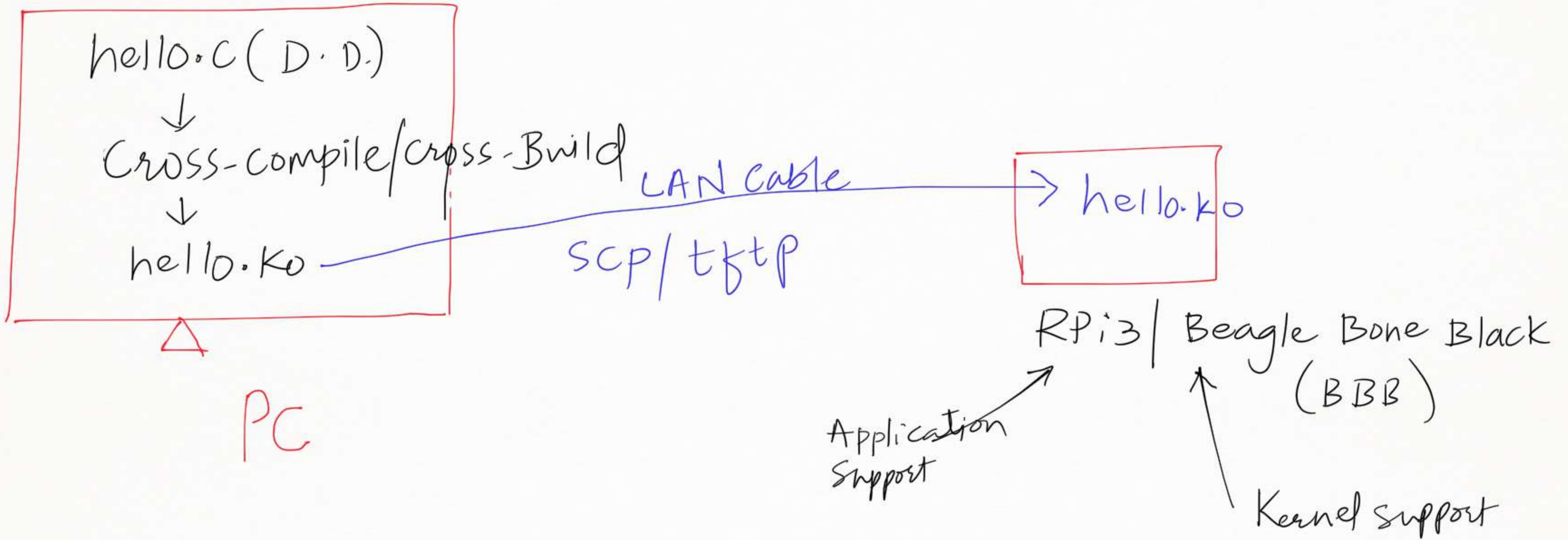


hello.ko

executed
hello.ko
in the same
PC



PC



O.S. Installation in RPi3

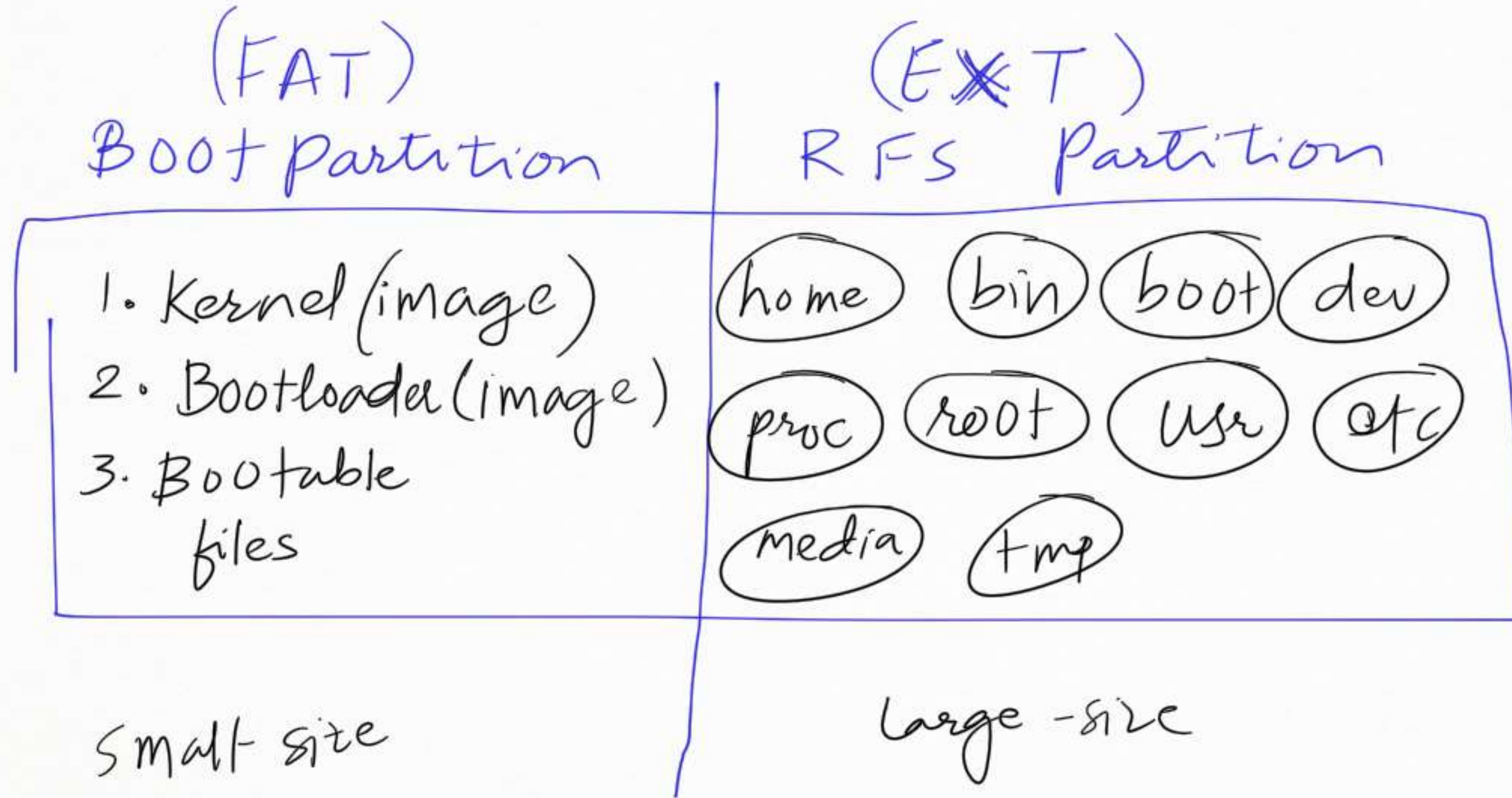
O.S?

↳ Raspbian O.S. ✓

↳ Build your own basic version of linux based O.S.

↳ Yocto tool "assignment"

↳ Buildroot tool



Micro
SD
Card

RFS: Root File System

Board Bring-up Images

init == systemd

1. **Bootloader** (search & load linux kernel into RAM)
2. **Kernel** (init h/w & O.S. parameter $\rightarrow$ creates user & kernel space)
3. **RFS** (commands, ---
/bin /sbin)

Desktop OS.

Bootloader: GRUB, LILO, NT
(name) (old) loader
(Windows)

Kernel: vmlinuz - X.X.X
Kernel version

RFS: /home /bin /boot /proc /dev

Embedded O.S.

Bootloader: Universal Bootloader
(u-boot), Red-boot
Recommended Not so popular

Kernel: zImage

RFS: /home /bin /boot /proc /dev

Assign.

Find the name RPi Kernel & Bootloader

O.S. Installation

R Pi/BBB

SBC ✓

lite version Raspbian O.S.



R Pi Imager tool



Installed lite v. Raspbian OS
on SD card

max 12

GPI O → LED

→ Push button

→ motor

Interrupt handling

Find out RPi3B Specifications

SOC → ?

Processor → ?

Cores → ?

Arch → ?

Audio → ?

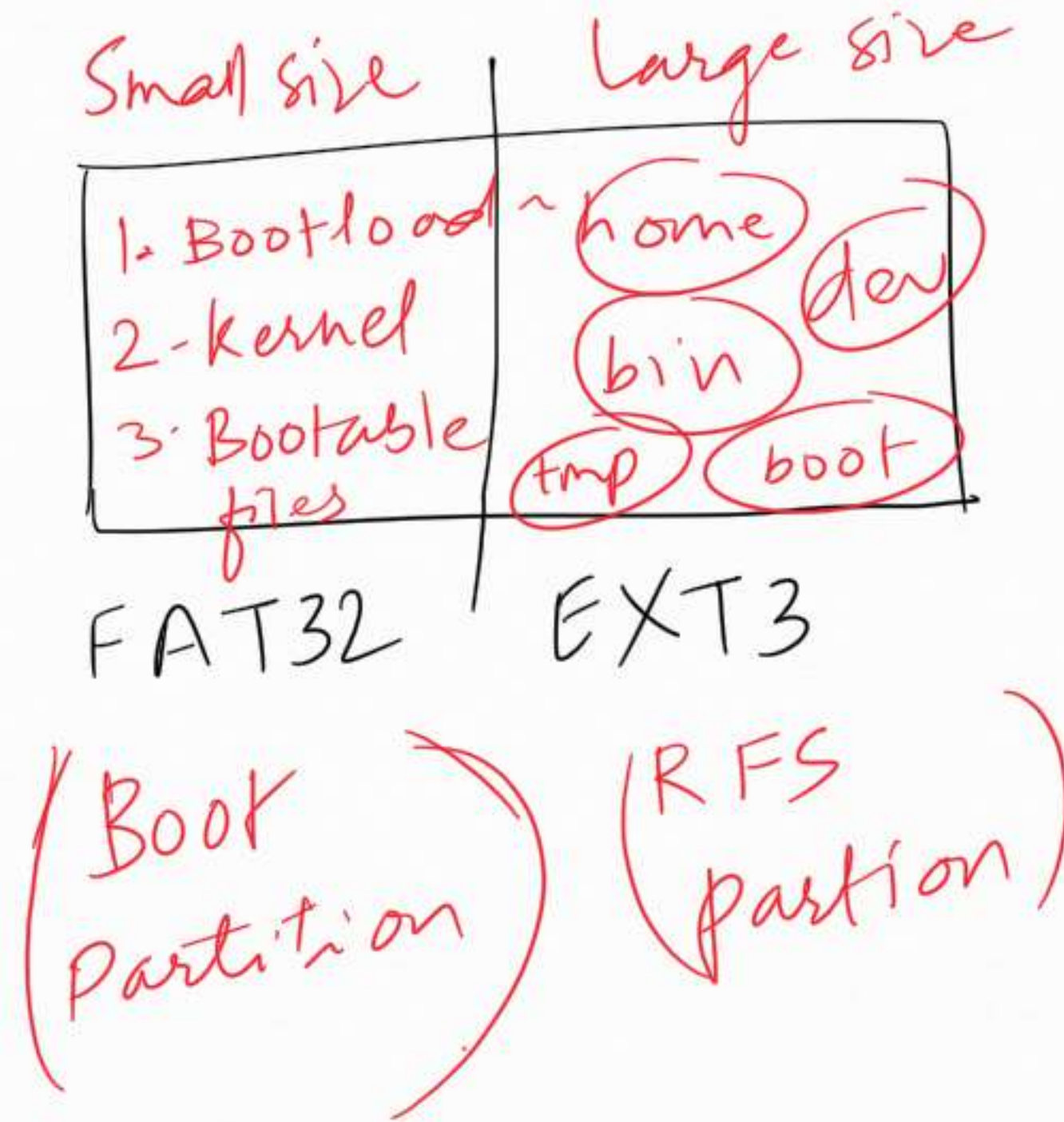
Video → ?

USB → ?

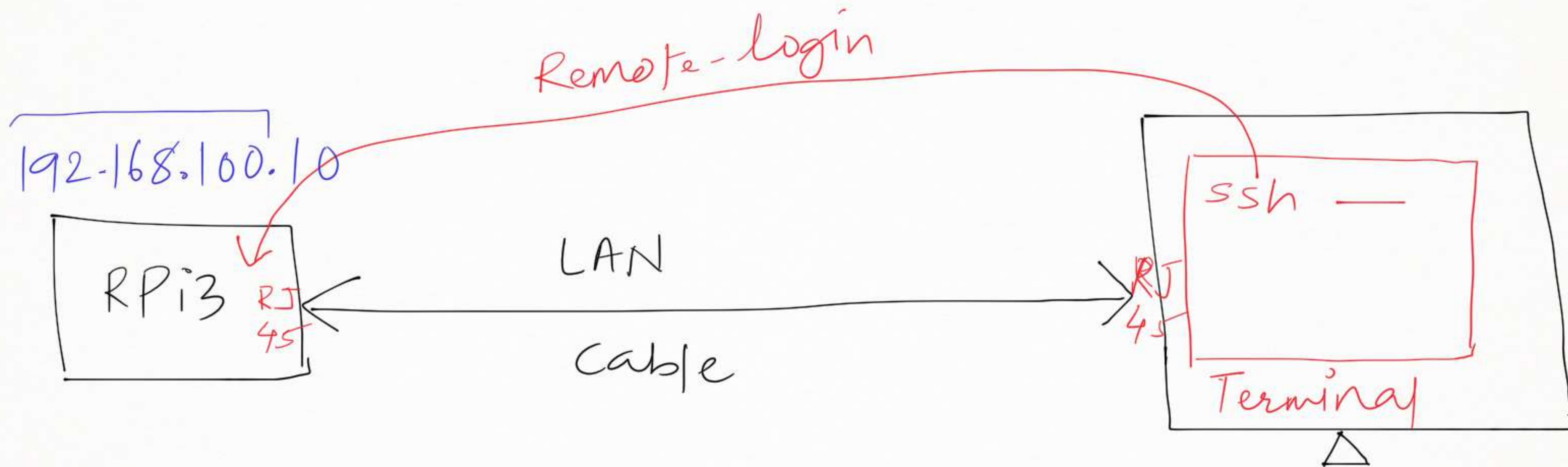
Wifi & BT → ?

Ethernet ?

Raspbian O.S.
(lite version)



8GB
memory card



ssh login

Credentials

↳ login name RPi

↳ Passwd

↳ IP Addr of RPi

Laptop

192.168.100.20

Login: pi

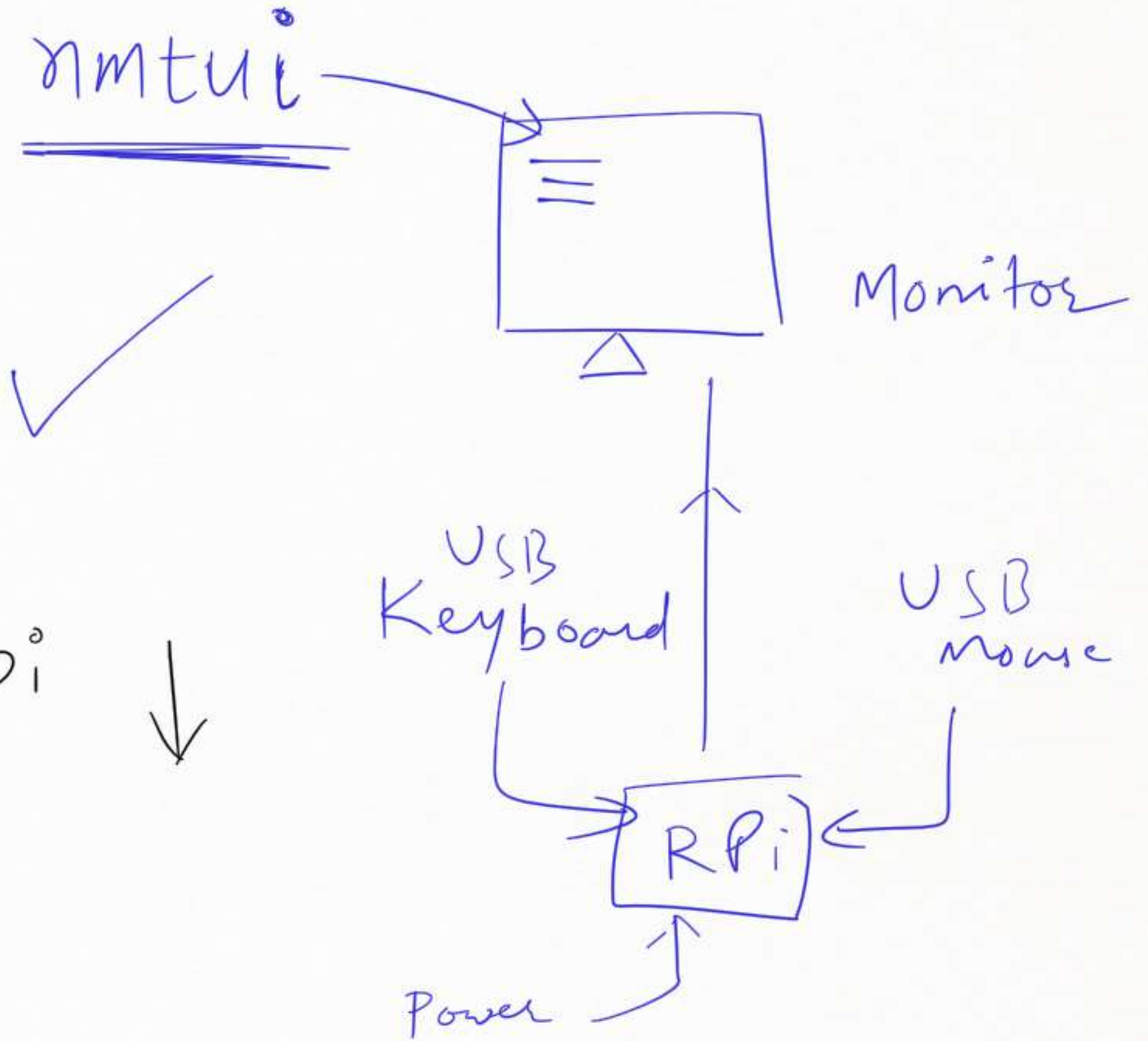
passwd: 123

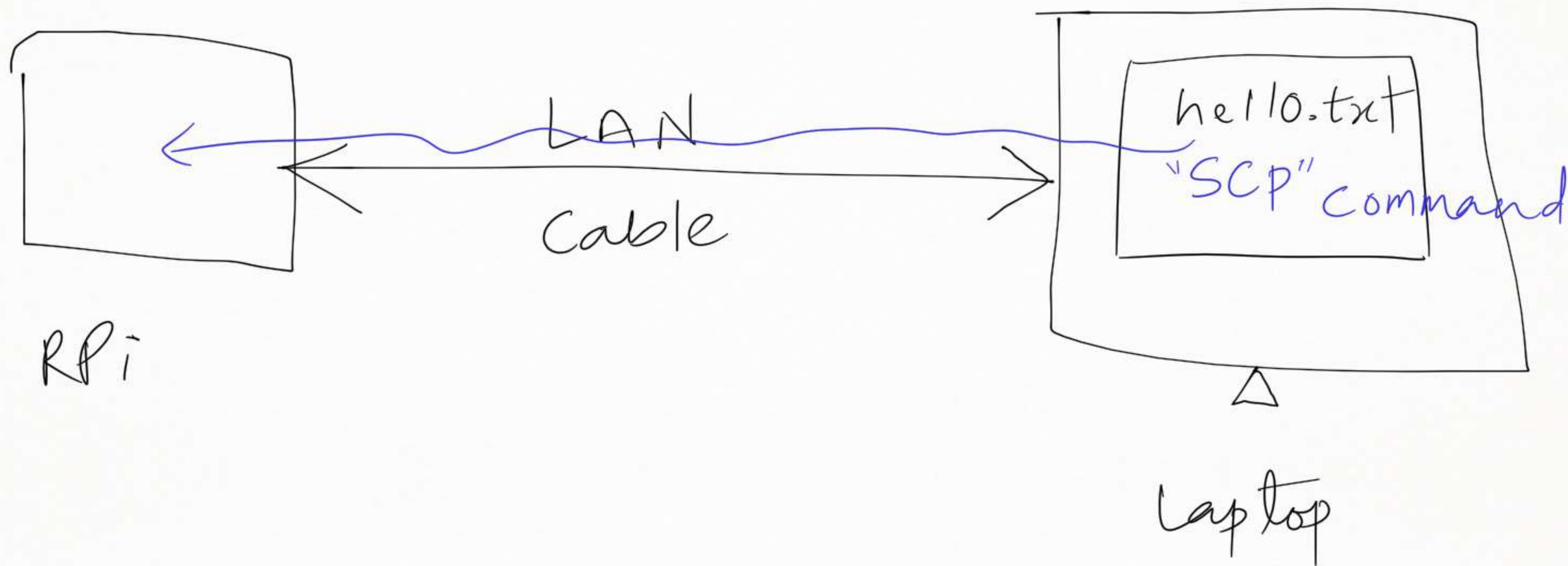
IP Addr: 192.168.100.10 ✓

ssh 192.168.100.10 -l pi ↓

passwd: 123 ↓

You are inside RPi.





Road map 1

1. RPi3B (H/w)

2. OS Installation

3. IP Addr setup in RPi & PC *

4. Remote login using ssh

5. Assign: file transfer PC $\rightarrow$ RPi using SCP

✓ 2pm - 4pm

✓ 5.30pm - 6.30pm

1. Linux Kernel ARCH

2. Linux file hierarchy (system)

↳ /bin

↳ /boot

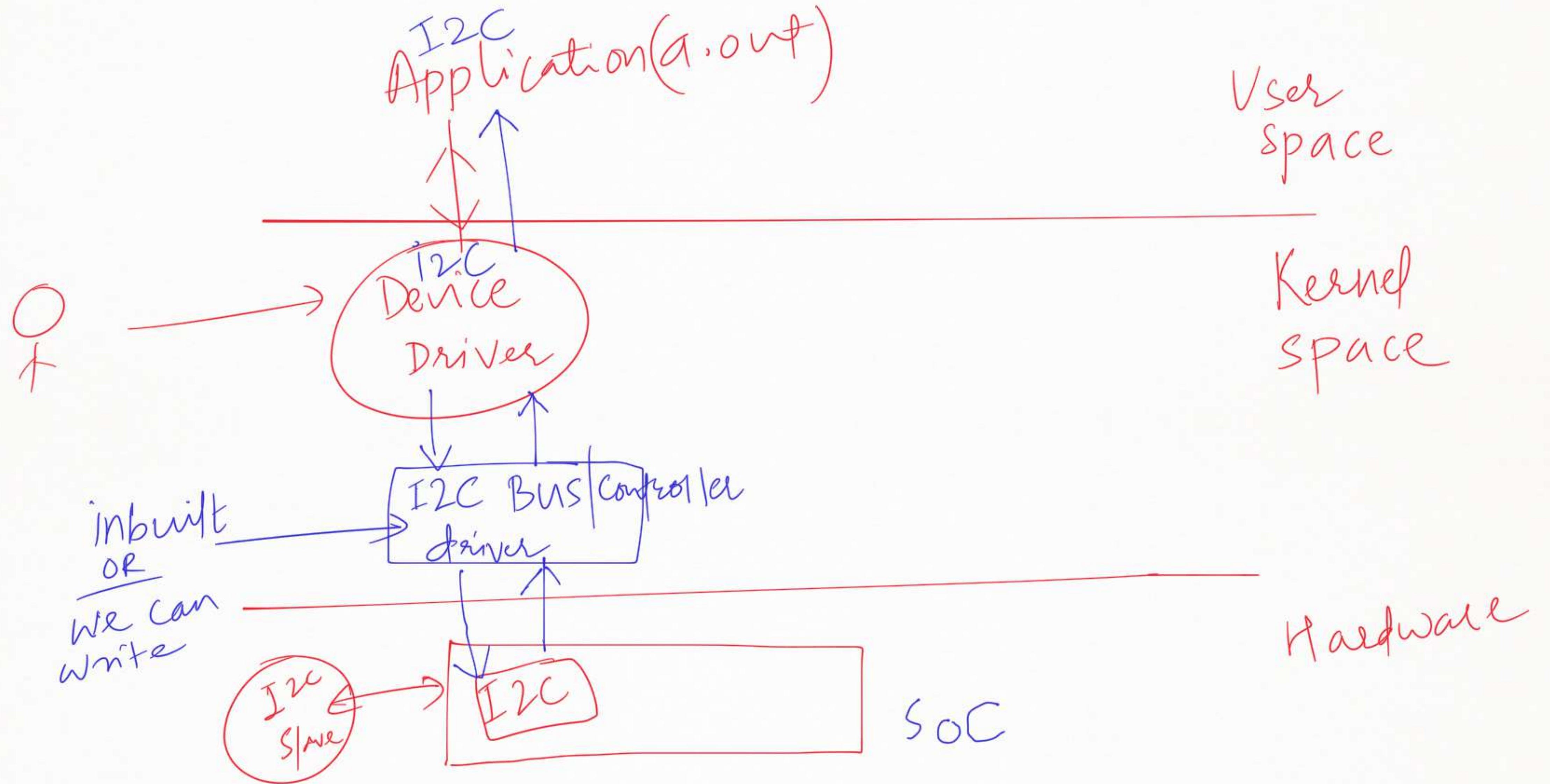
↳ /dev *

↳ /media

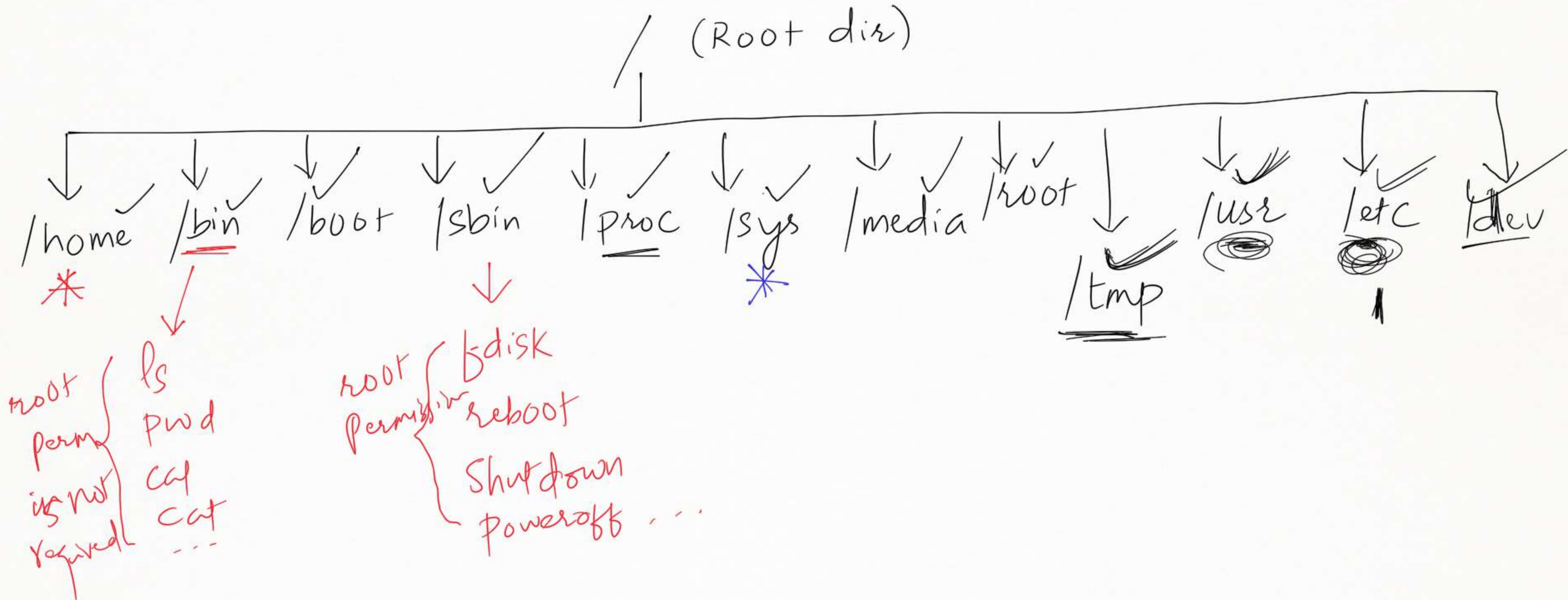
↳ /tmp

↳ /sys *

↳ /proc *

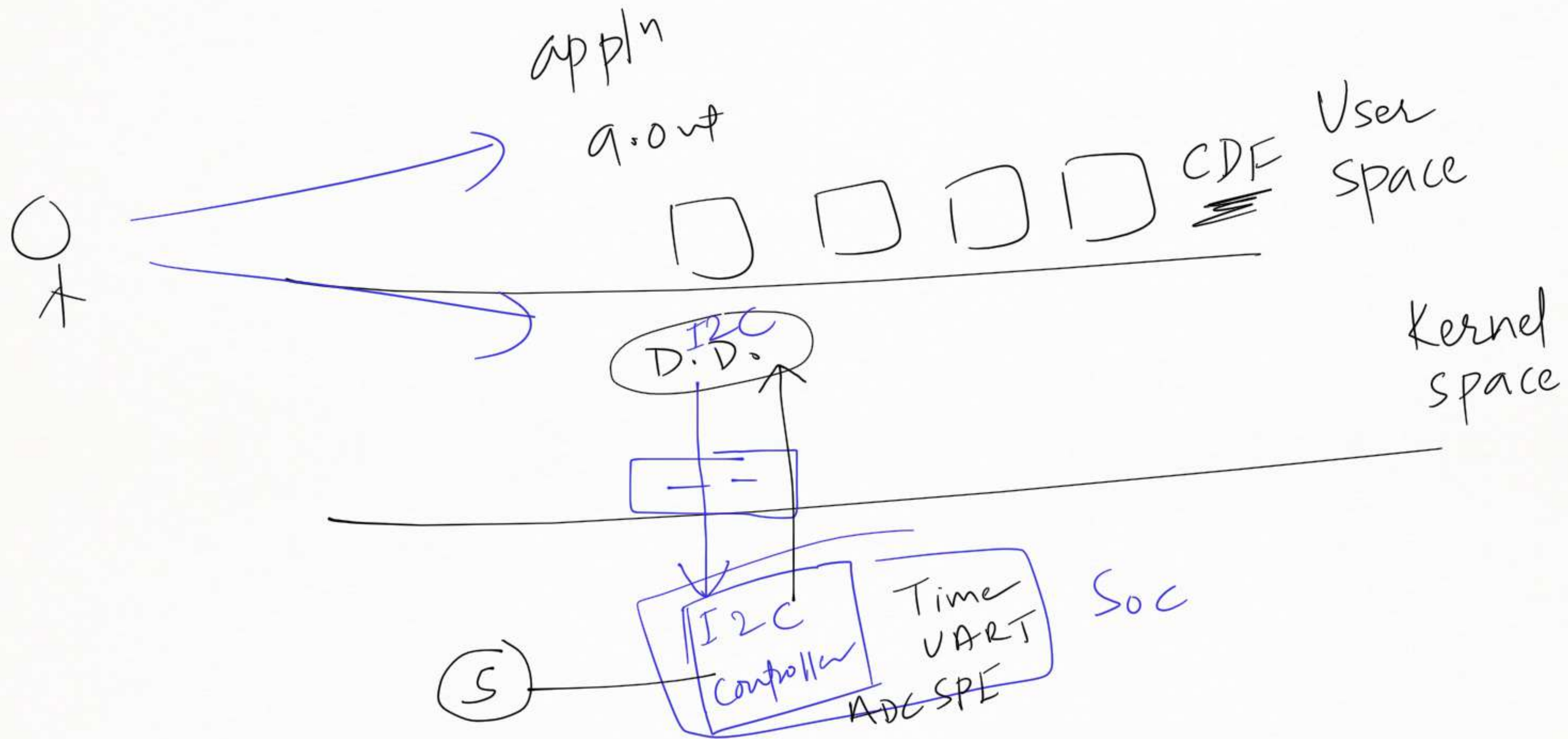


Linux File System Structure



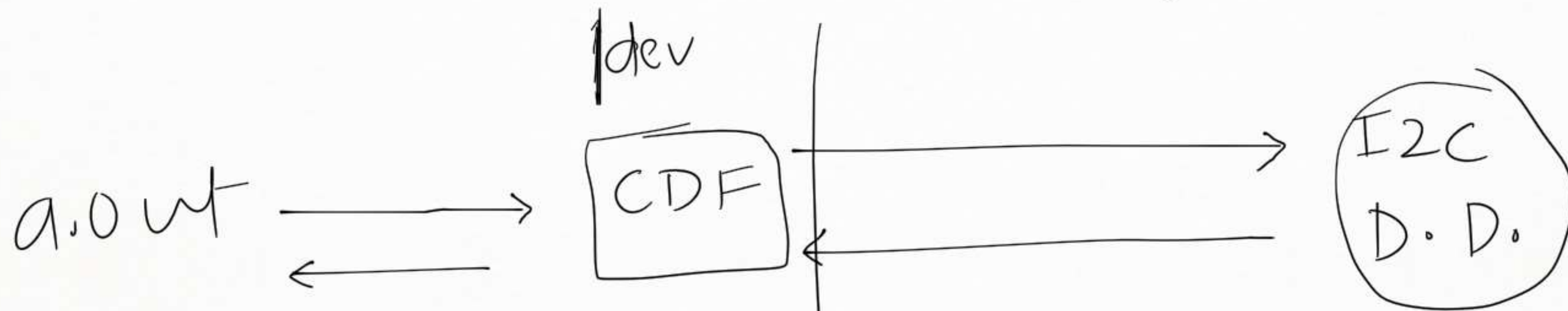
/dev

1. character device files (Keyboard, mouse, monitor, i2c, spi)
2. block device files (SSD, HDD, RAM)



• /a.out

↳ I2C sensor output for every 1sec



```
fd = open("/dev/i2c0", O_RDWR);
```

User Space

1545

↳ class → ~~GPIOR2~~ → value
new



Export GPIO25 $\xrightarrow{1/0}$ Led

Kernel space

$$H/\omega$$

I2C drive

GPIO driver

New drive

1. Toolchain download & install
2. Cross-compile linux kernel
(Kernel compilation) } 1/2 hrs
3. Cross-compile driver
4. Cross compile app/ln

Toolchain

↳ Cross-compile "driver" & applications

Download toolchain for RPi

github.com/raspberrypi/tools

Assignment

Cross-compile

C-programs & execute
in RPi

Do Same for Linux-programs

TCP/IP programs.



Ubuntu sys

- o hai → cross-compiled
for RPi

How to cross-compile c-program? ✓

How to cross-compile device-driver for RPi3?

check Kernel
version of RPi
→ 6.6.51

RPi3

hello.c (driver)



Cross-compile



hello.ko (modinfo hello.ko)

↳ check Kernel version

hello.c



compile



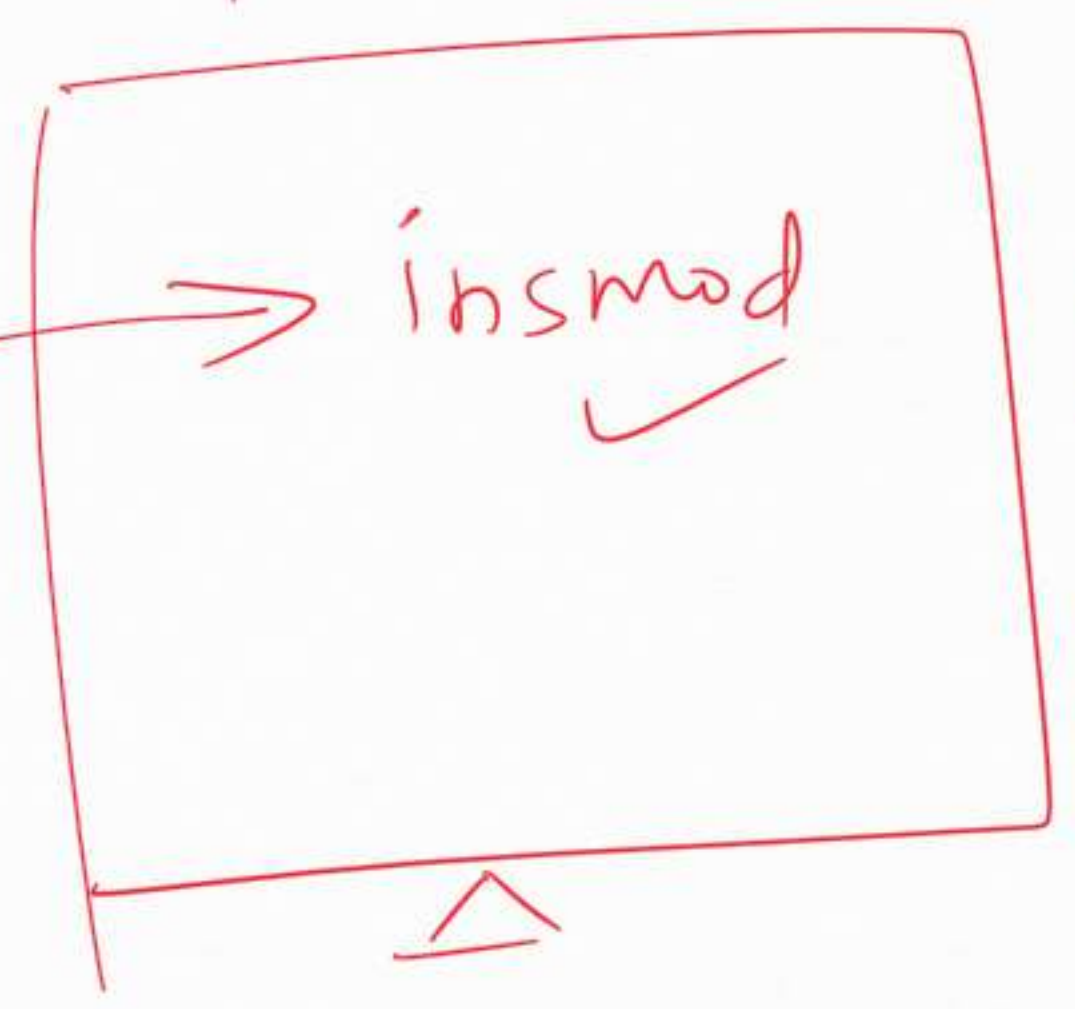
hello.ko

(5.12.0)

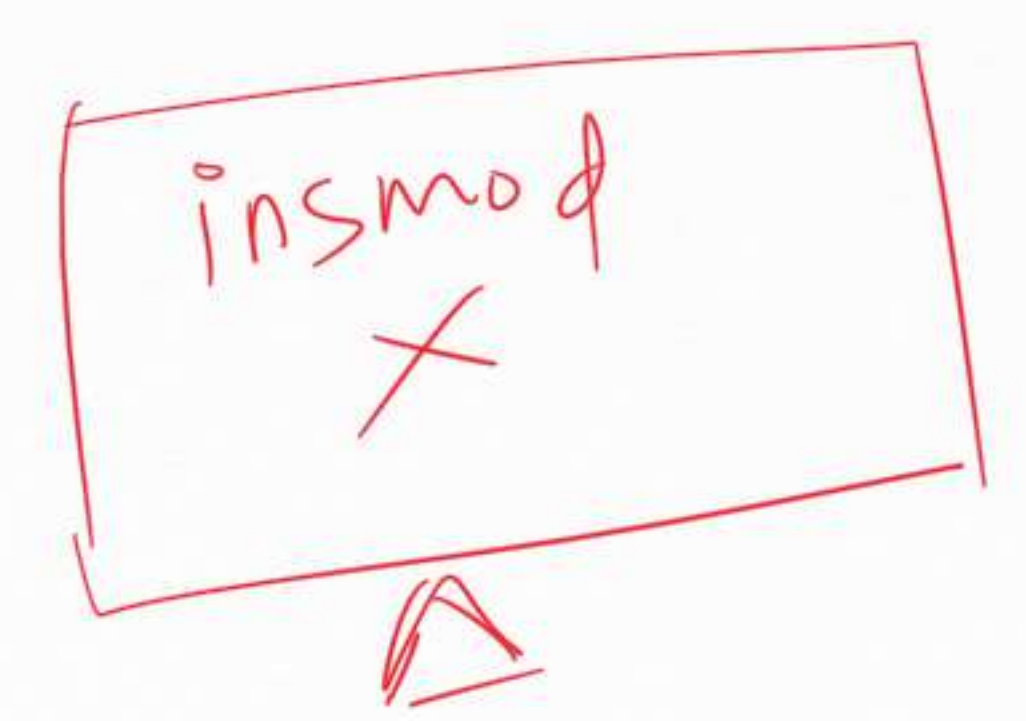


Kernel version

PC/RPi: 5.12.0



①



PC ②

5.13.2

You

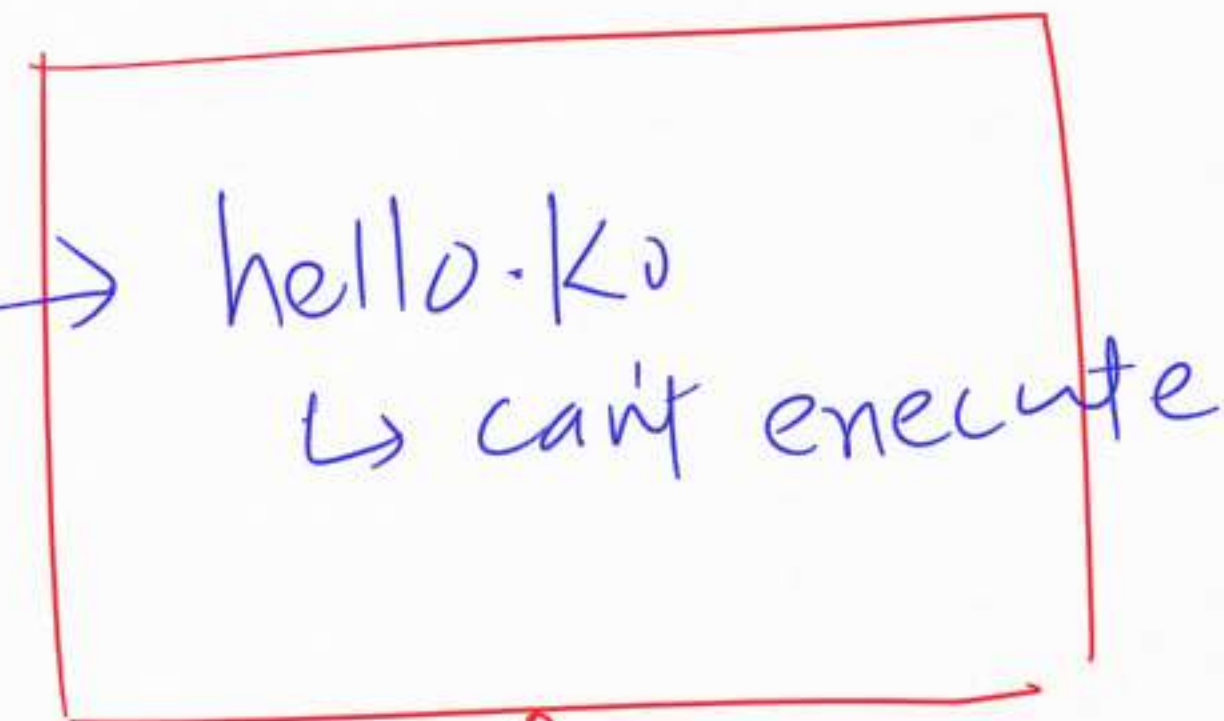


PC

Kernel version : 5.12.4

Ubuntu 20.04

Client



Kernel version : 4.10.2

Ubuntu 20.04

1. Kernel compilation

2. Kernel porting

Kernel porting

- Additional functionalities
- Adding/Removing existed drivers
- Reduce size/faster execution

Download Kernel source code



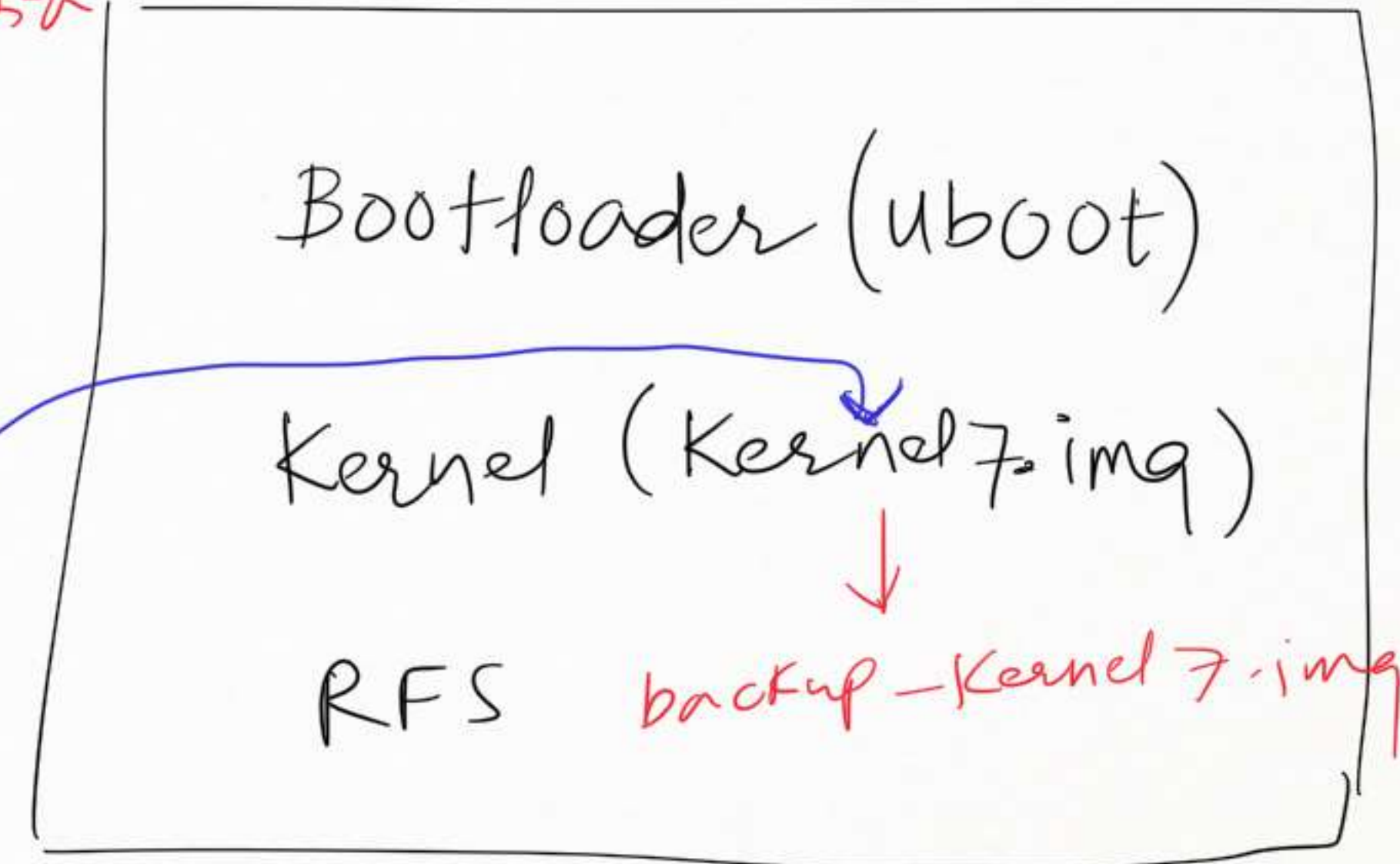
Kernel Customisation



Kernel Compile

↓ 1/2 hr / 1+hr

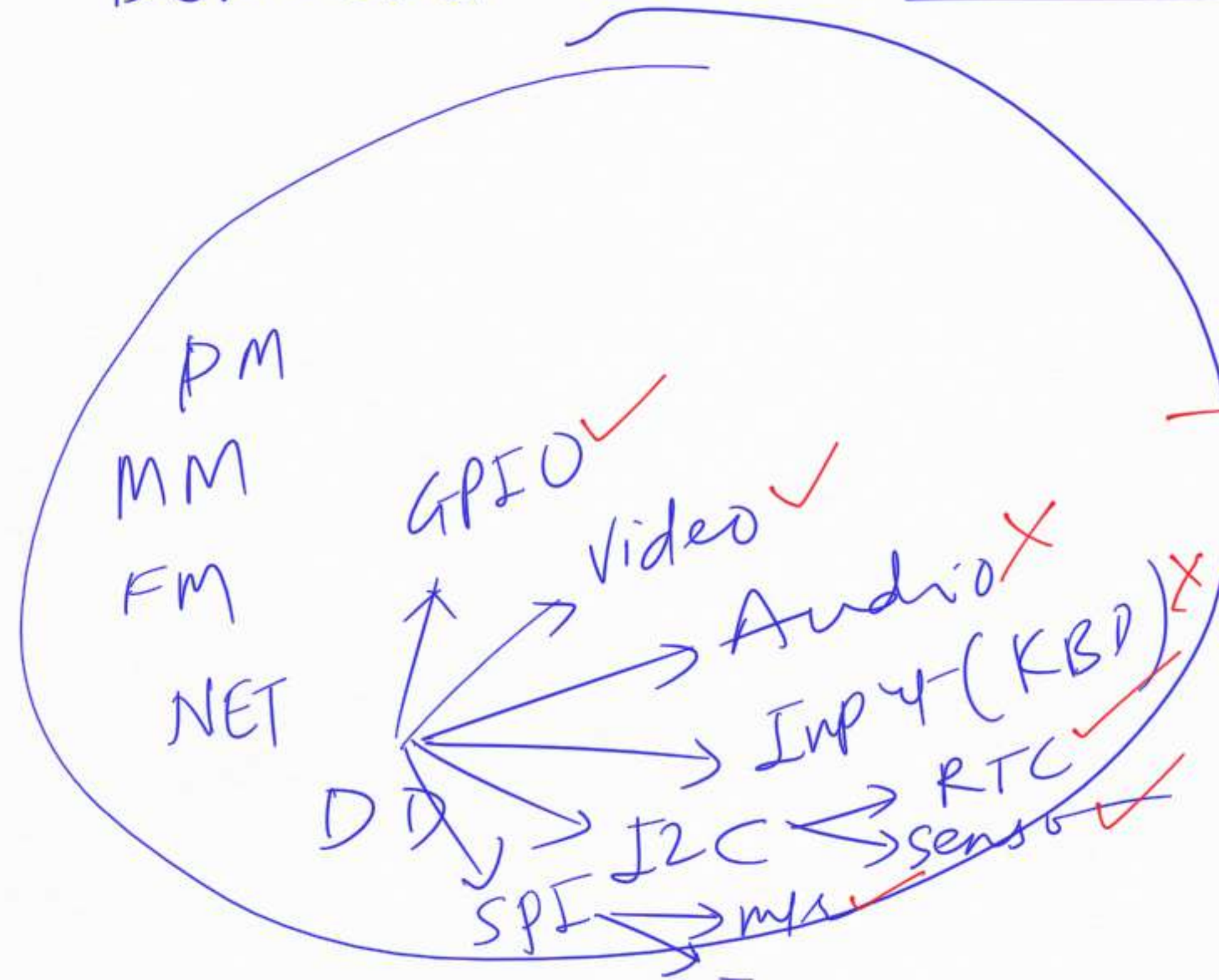
zImage (kernel image)



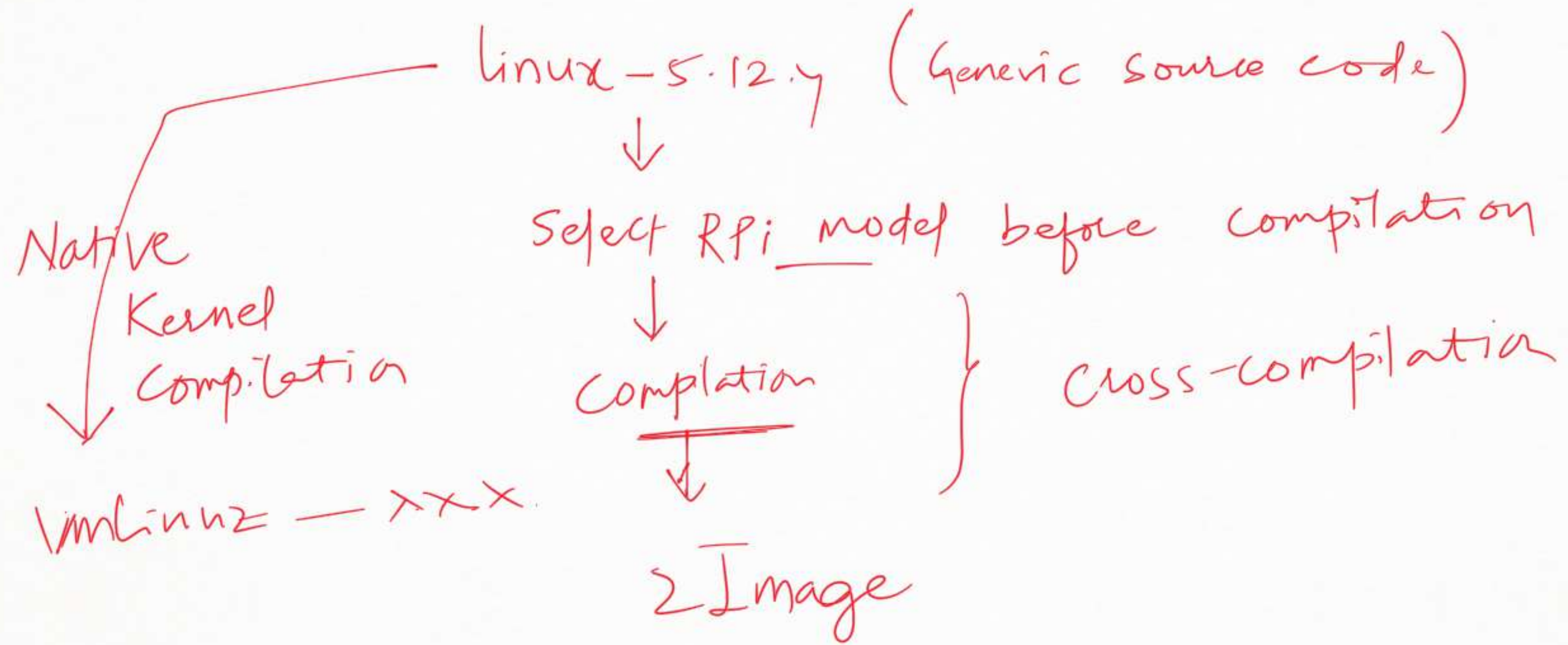
RPi (SBC)

Download Kernel Source Code

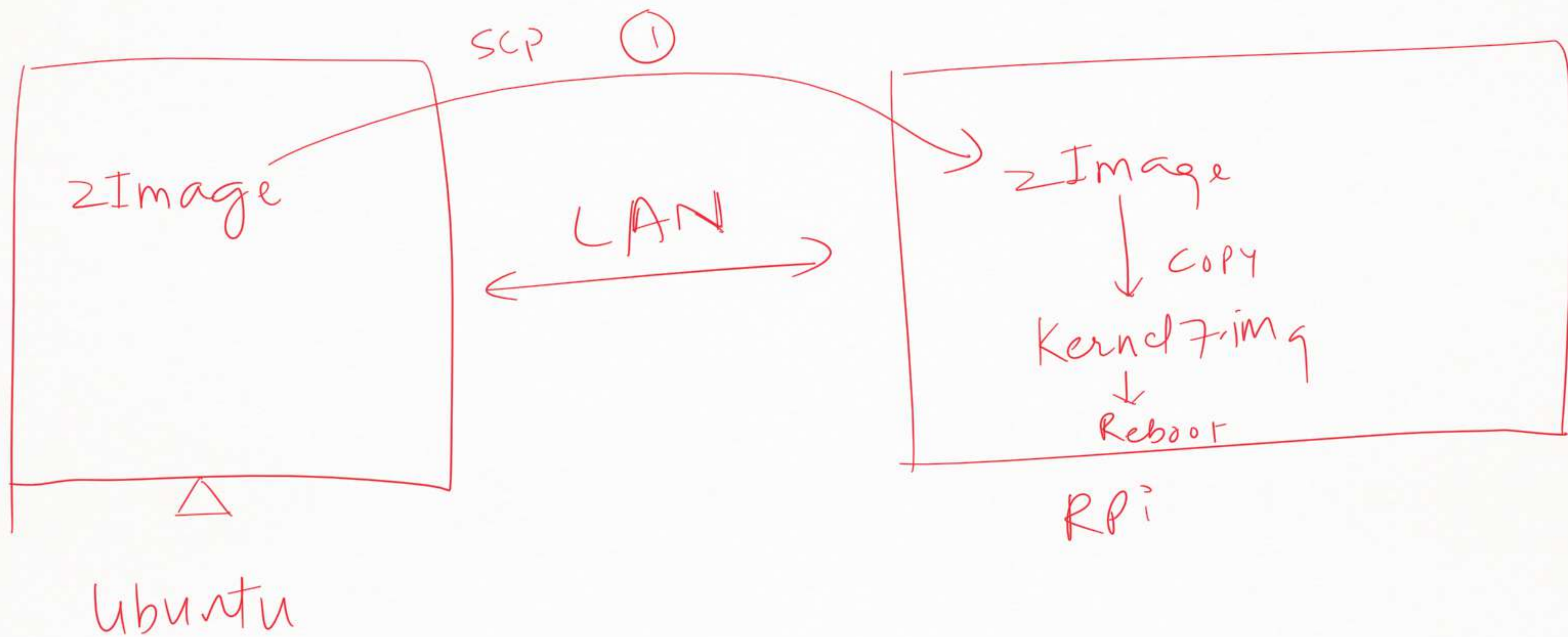
↳ POS(ubuntu)
↓
vmlinuz - X.X.X
Version

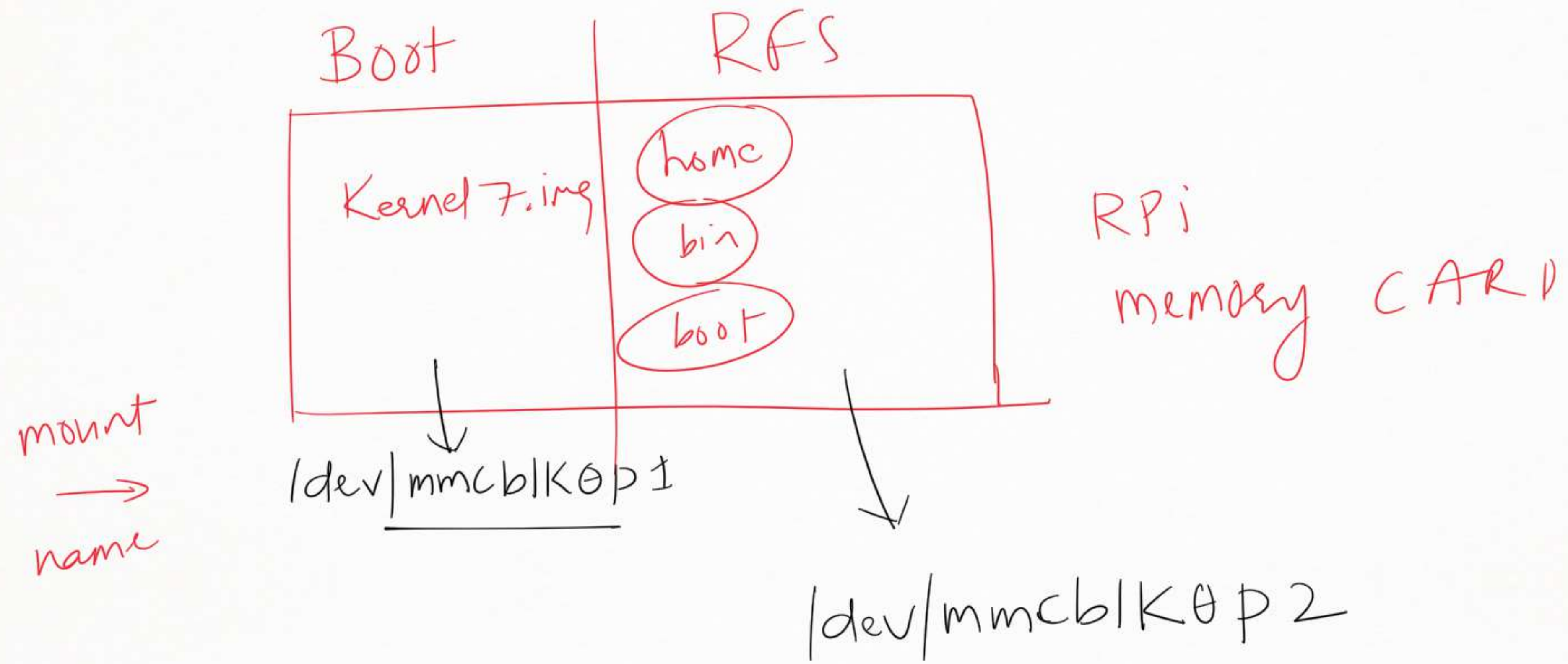


→ "zImage"
Monolithic



Setup Cross-compilation Env. in ~~your~~ Ubuntu
Terminal





In RPi

\$ sudo mount /dev/mmcblk0p1 /tmp

\$ cd /tmp

\$ sudo cp kernel7.img

back up - kernel7.img

\$ sudo cp ~/zImage

./kernel7.img

& reboot Pi

\$ sudo umount /tmp
\$ cd

~~GPIOL~~

I2C

SPI

CAN

Character device driver.

"register"

U.S.

♀ cdd.ko

insmod

syscal interface

K.S.

rmmod

"unregister"

cdd.ko

H/W

API (lib. funcs)

1. register_chrdev
2. unregister_chrdev

Major Num: class of device

Minor num: represents unique device under class

eg: USB (class of device)

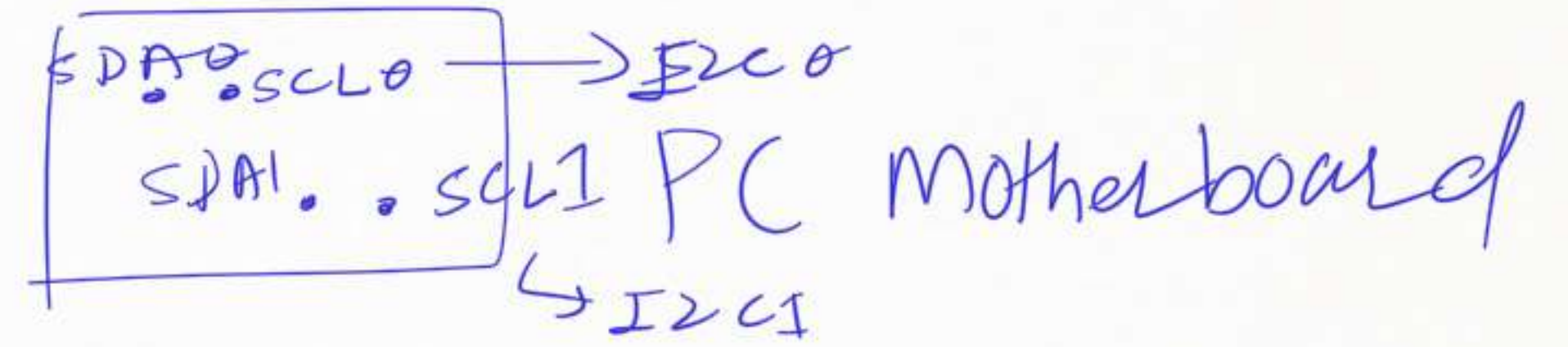
major minor
100 / 0 USB port 1 device

100 / 1
100 / 2 USB port 3 device



I2C (class of device)

	<u>Major</u>	<u>minor</u>
I2C-0	→ 250,	0
I2C-1	→ 250,	1



○ 250,0 (i2c0 bus)
↑
Kernel 250,1 (i2c1 bus)

List of devices & major nums

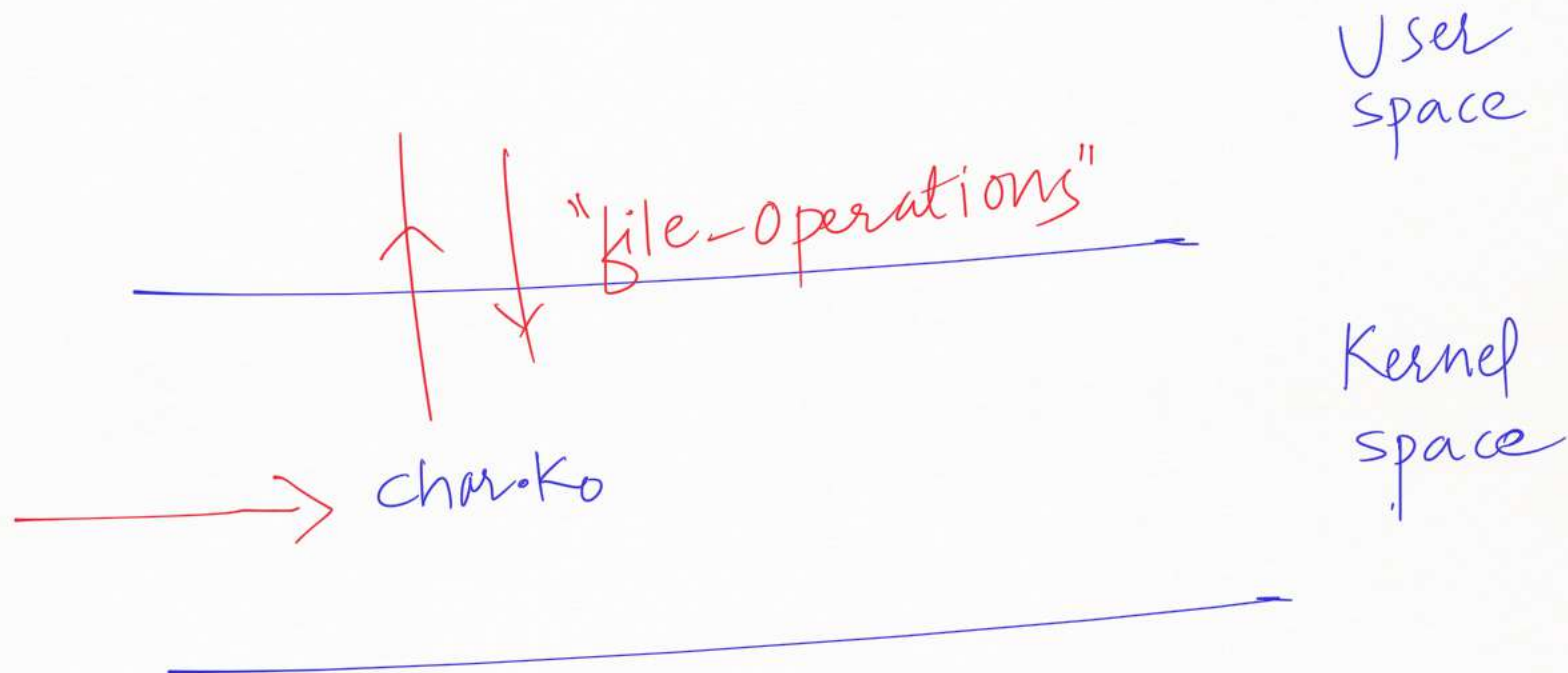
1. ls -l /dev

2. cat /proc/devices

Major num - 12bits

minor num - 20bits

struct file-operations



struct file_operations {

(*open)
(*release)
(*write),
(*read),
(*ioctl),
...
}

} ~~file~~ pointers
function

fs.h

Path of Kernel-space headers files

/lib/modules/kernel-version/build/

include/linux

→ fs.h

Path of user space headers files

/usr/include

Your driver

```
my_write(-){  
    }
```

```
my_read(-){  
    }
```

```
struct file_operations fops;  
fops.write = my_write;  
fops.read = my_read;  
fops.open = my_open;  
fops.release = my_release;
```

User space

C-lang prog

main() {

open

write

read

close

}

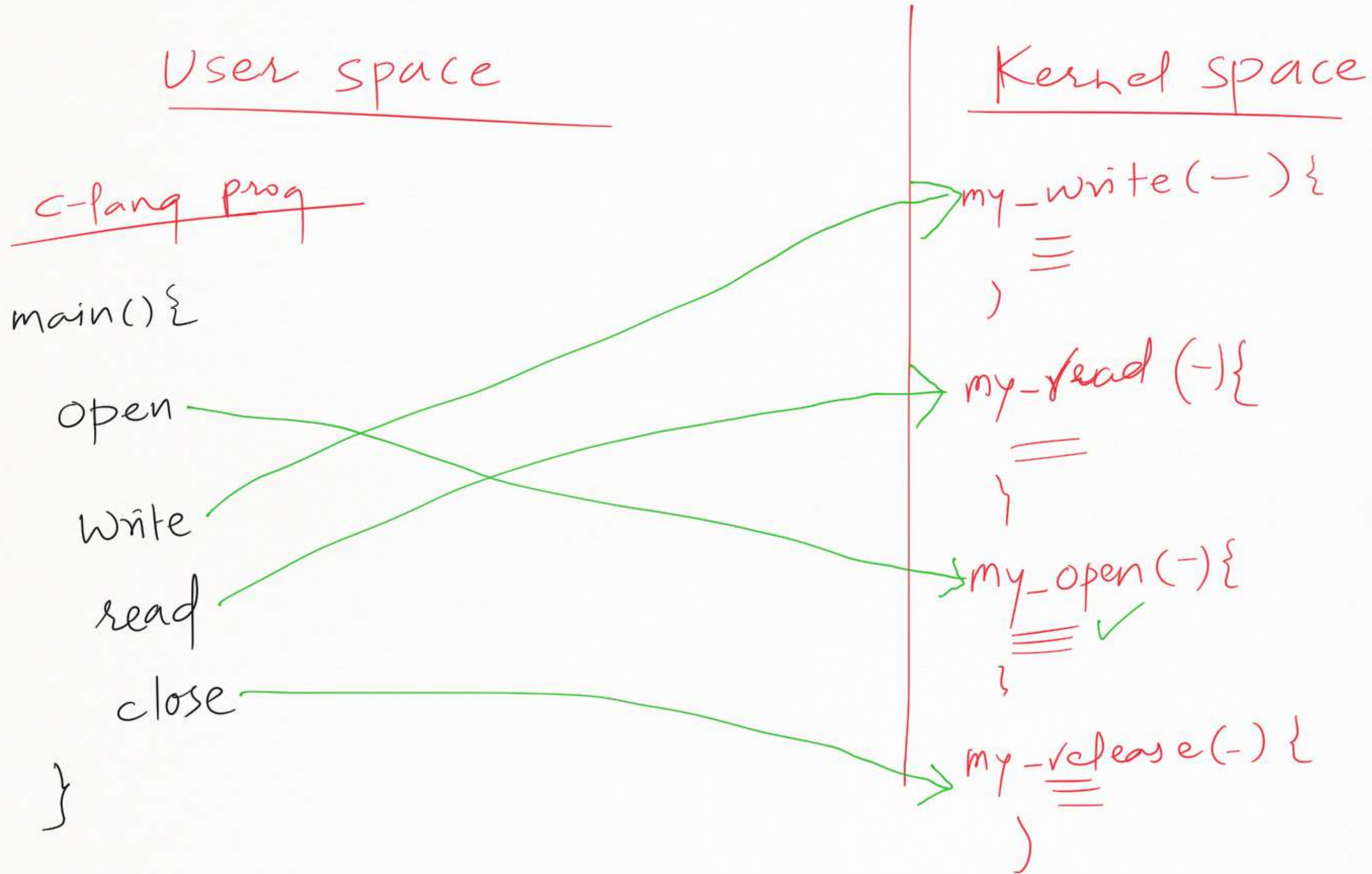
Kernel space

my_write(-) {
==
}

my_read(-) {
==
}

my_open(-) {
== ✓
}

my_release(-) {
==
}



User space

C-lang prog

```
main() {
```

```
  open
```

```
  write
```

```
  read
```

```
  close
```

```
}
```

/dev/myfile

char device
file

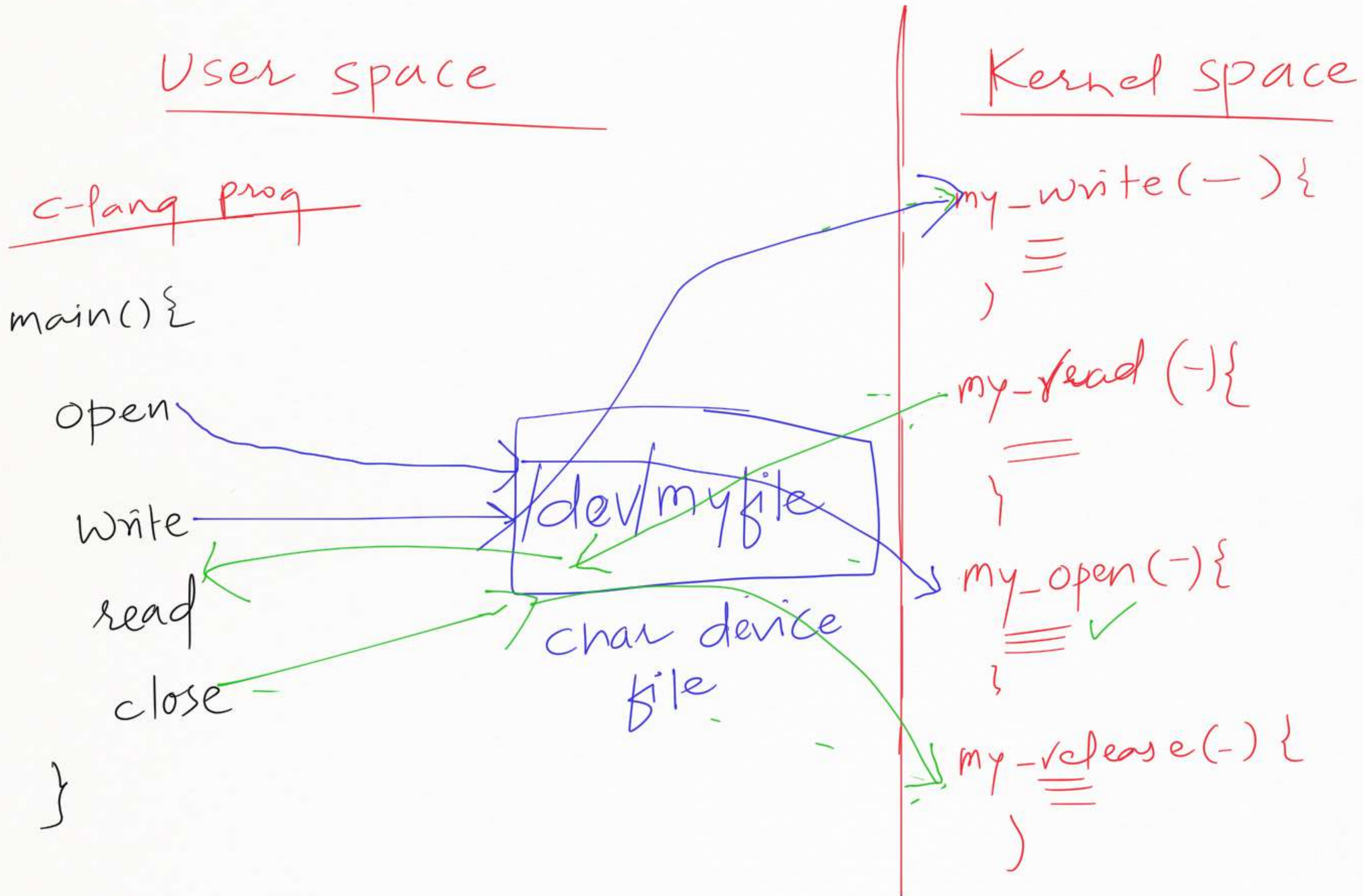
Kernel space

```
my_write(-) {  
    ==  
}
```

```
my_read(-) {  
    ==  
}
```

```
my_open(-) {  
    == ✓  
}
```

```
my_release(-) {  
    ==  
}
```



Assign:

1. How to create CDF using command?
2. How user space system calls are mapped with Kernel space Bunks?

User space

```
main() {
```

```
  open
```

```
  write
```

```
  read
```

```
  close
```

```
}
```

/dev/myfile

CDF

Kernel space

```
my-open() {
```

```
  ≡
```

```
my-write() {
```

```
  ≡
```

```
my-read() {
```

```
  ≡
```

```
my-release() {
```

```
  ≡
```

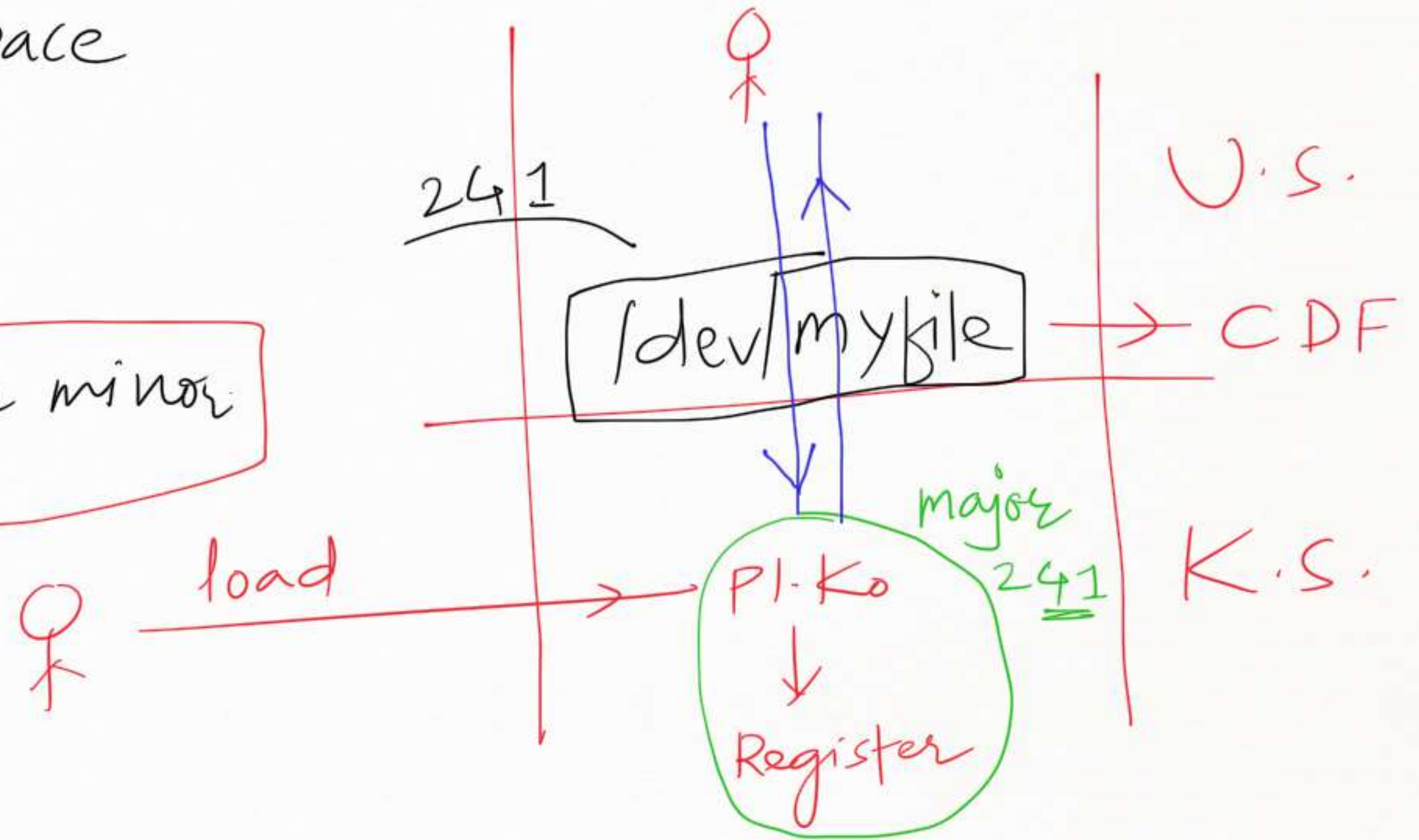
```
}
```

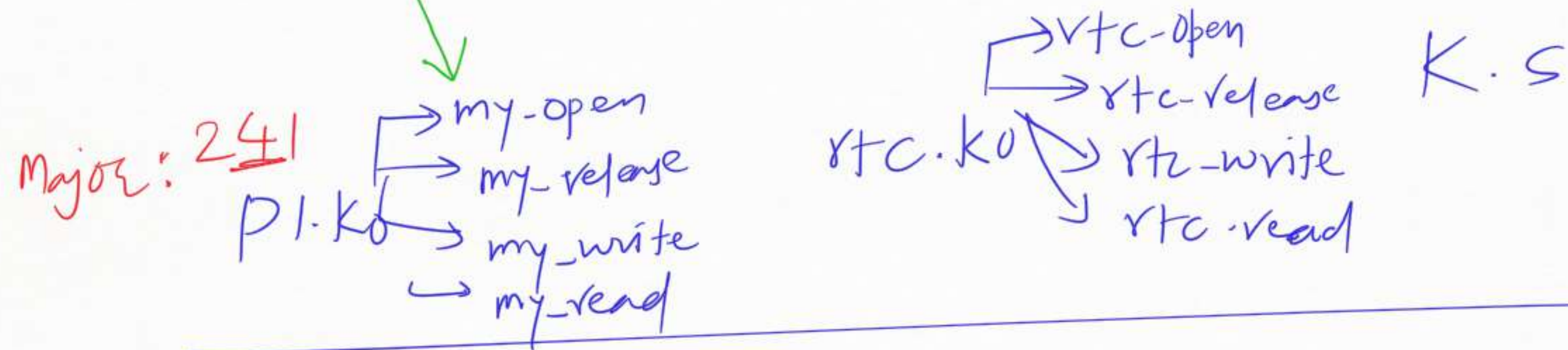
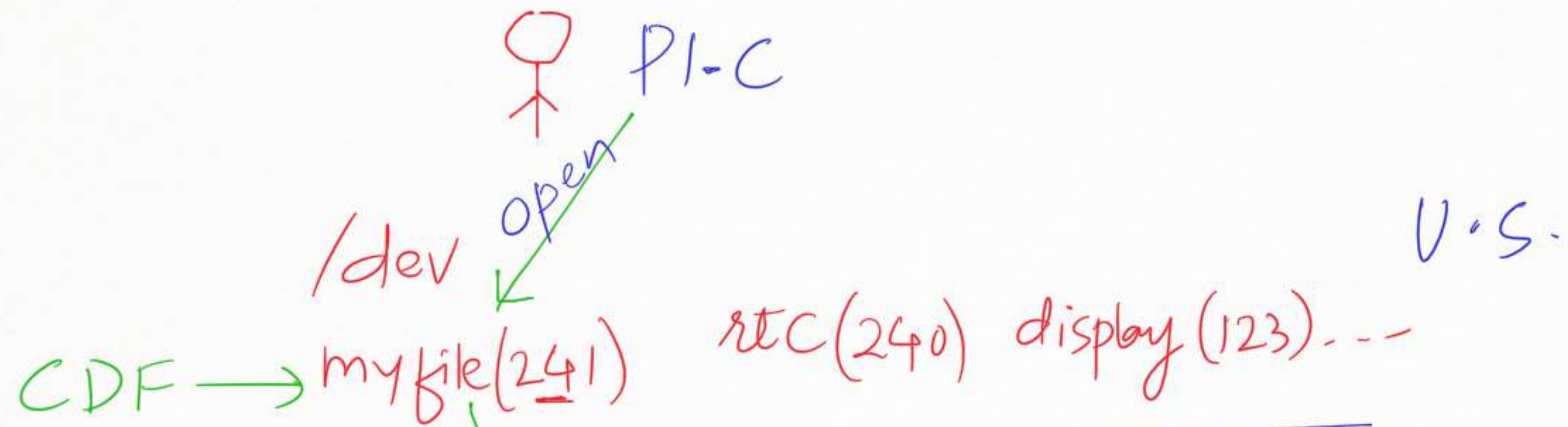
mknod → Create CDF in user space

Syntax

`sudo mknod /dev/file_name c major minor`

`cat /proc/devices`





Major	(*open)	(*release)	(*write)	(*read)	*ioctl
241	my-open	my-release	my-write	my-read	my_ioctl
240	rtc-open	rtc-release	rtc-write	rtc-read	rtc_ioctl
123	-	-	-	-	-

System
call

Interface

led.c

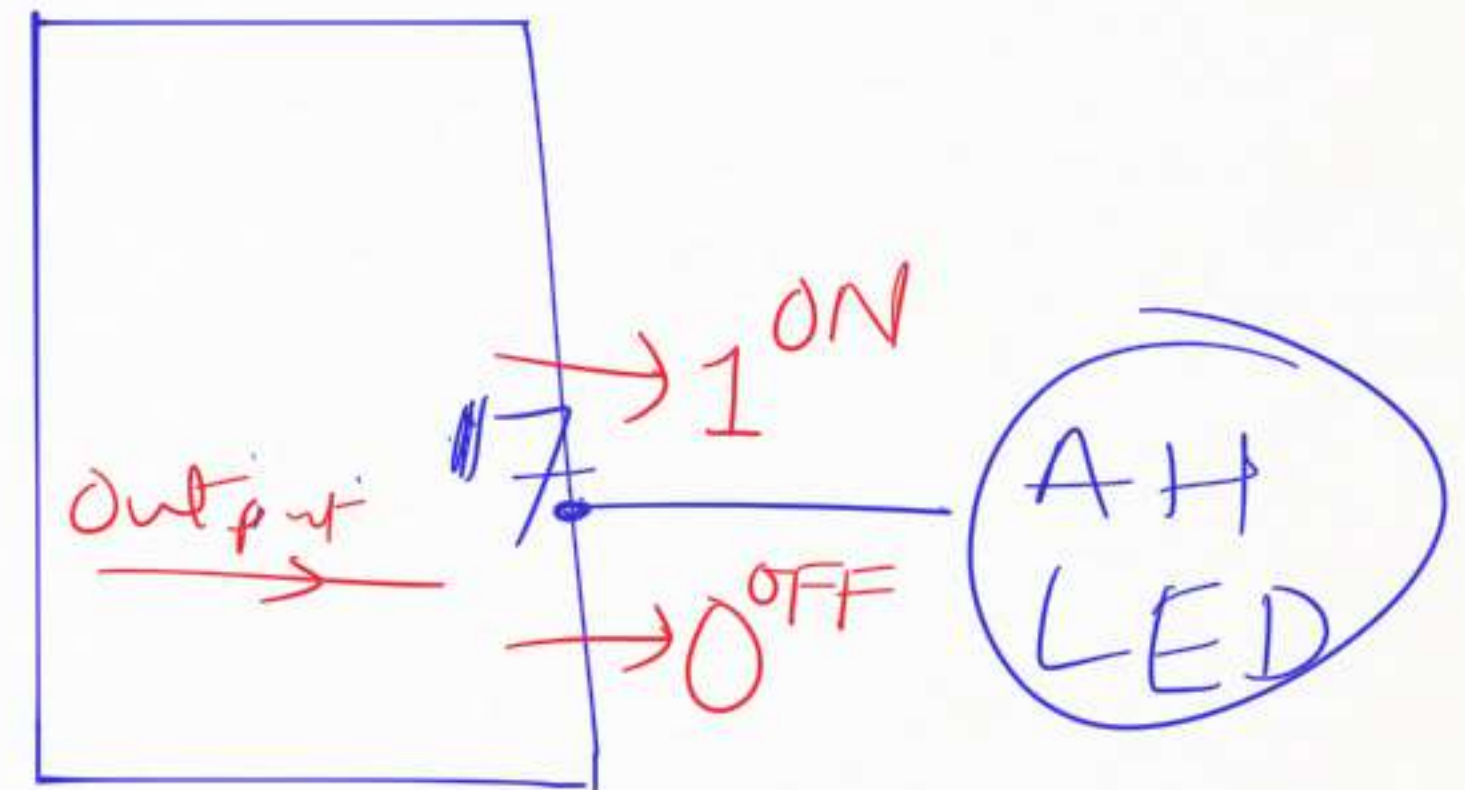
init() { 0. gpio-free ^{→ rmmmod}

1. gpio_request * ← insmod

2. gpio-direction

3. gpio-set-val → led, motor, display - - -

4. gpio-get-val → sensor, switches



init_led() {

1. register - chdev

2. gpio_request

3. gpio_direction

4. gpio_set_value (high)

↳ LED ON

}

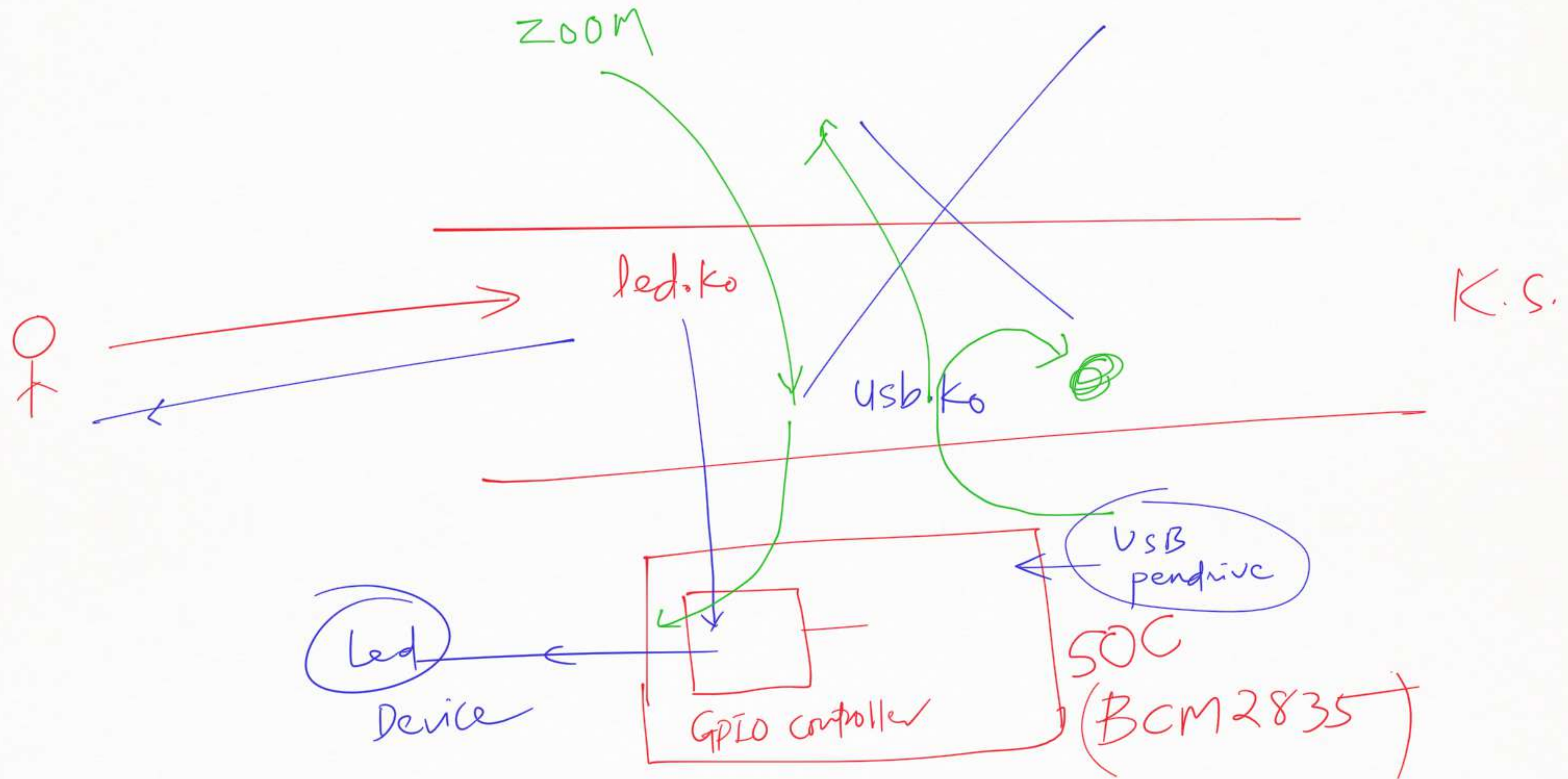
exit_led() {

1. unregister - chdev

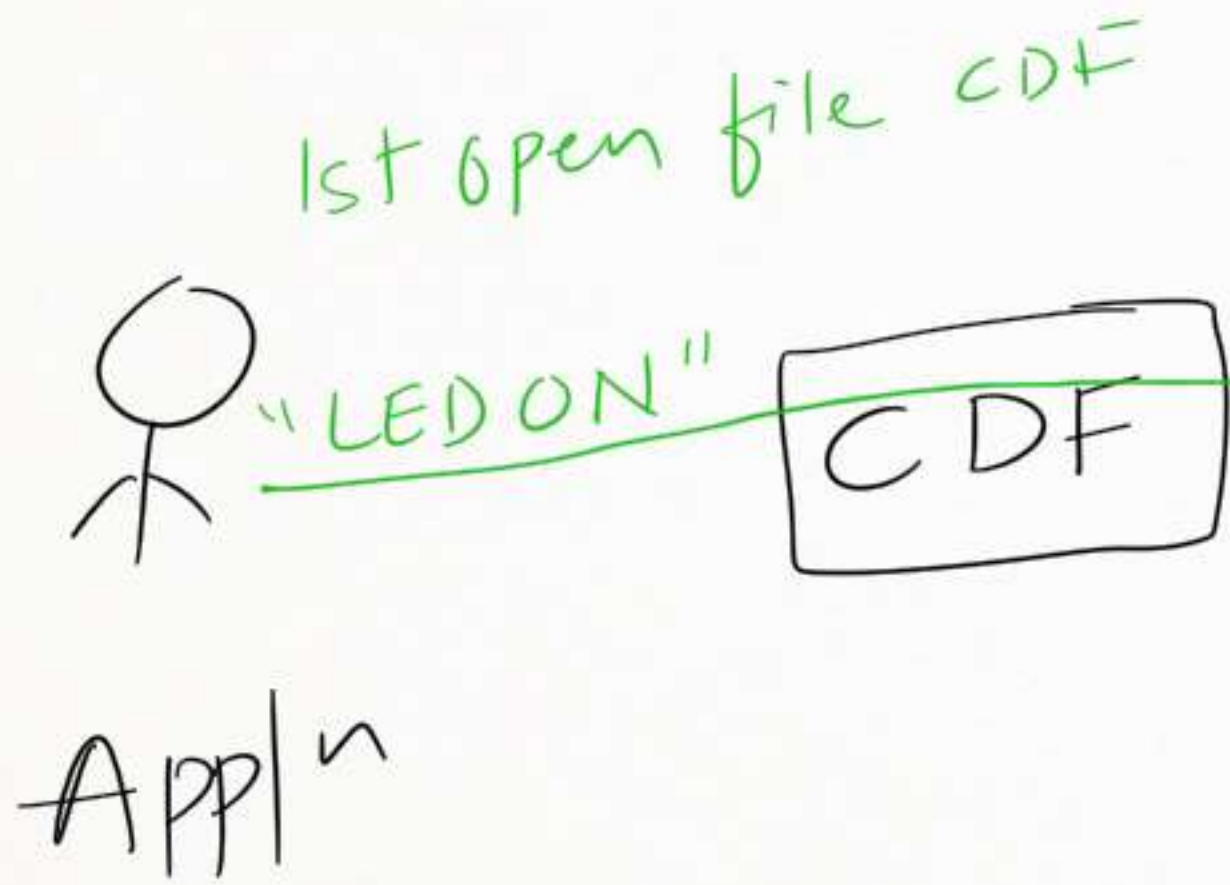
2. gpio_set_value (low);

3. gpio_free ↳ LED OFF

}



User Space

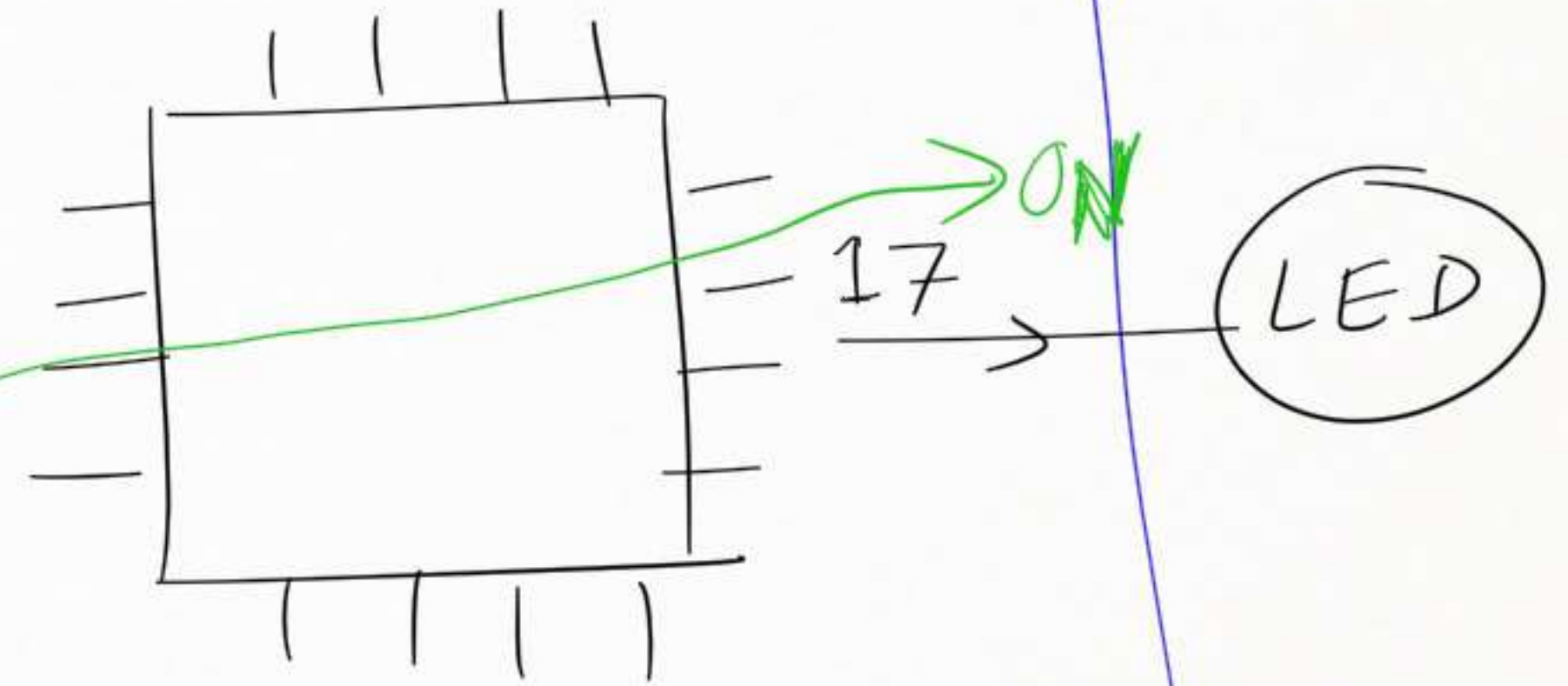


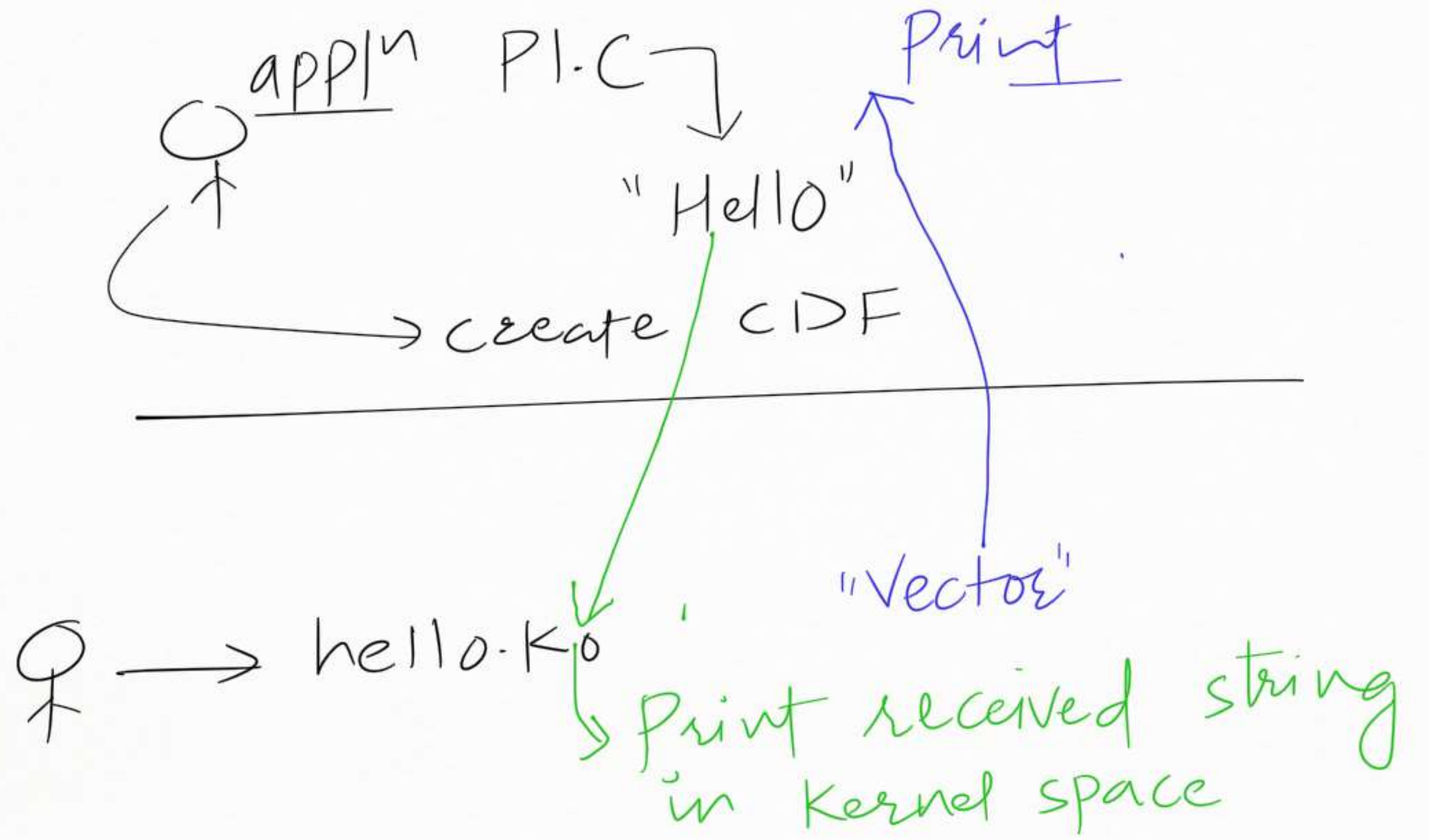
Kernel space

request

```
your_write() {  
    strcmp(—, —)  
    if (r == 0)  
        gpio_set_value(ON)  
  
    else  
        gpio_set_value(OFF)  
}
```

H/w space





Assignment
execute in
your PC
Native
compilation